

Using Tcl With Synopsys Tools

Version X-2025.06-SP3, March 2026



Copyright and Proprietary Information Notice

© 2026 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys, Ansys, the Synopsys and Ansys logos, and other Synopsys trademarks are available at <https://www.synopsys.com/company/legal/trademarks-brands.html>. Other company or product names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	8
Related Products, Publications, and Trademarks	8
Conventions	9
Customer Support	10

1. Introduction to the Tool Interfaces	11
Tcl and Synopsys Tools	11
Tool Interfaces	12
Starting the Command-Line Interface	13
Using Setup Files	14
Including Tcl Scripts	16
Using Command Log Files	16
Using the Filename Log File	17
Interrupting Commands	18
Controlling Information, Warning, and Error Messages	18
Running Linux Commands Within the Tool	19
Exiting the Tool	21

2. Commands	22
Command-Line Editor	22
Application Command Syntax	22
Special Characters	23
Wildcard Character	24
Creating Comments	24
Data Types	24
Strings	25
Lists	26
Arrays	27
Operators	29
Abbreviating Command and Option Names	30
Using Aliases	31

Contents

Multiple Line Commands and Multiple Commands per Line	32
Outputting Data to the Screen	32
Command Parsing	33
Substitution	34
Quoting	35
Special Characters	35
Bus Notation	36
Redirecting and Appending Command Output	37
Using the Redirection Operators	37
Using the redirect Command	37
Command Status	38
Successful Completion Example	38
Unsuccessful Execution Examples	38
Listing and Rerunning Previously Entered Commands	38
Getting Help on Commands	39
Using the help Command	40
Using the man Command	40
Tcl Limitations Within the Command-Line Interface	41
Basic Tcl Commands	41
cd and pwd	42
file and glob	42
open, close, and flush	43
gets and puts	44
Nonsequential File Access	45
 3. Variables	 46
Components of a Variable	46
Application Variables	46
Considerations When Using Variables	47
Manipulating Variables	48
Listing Existing Variables	48
Displaying Variable Values	49
Assigning Variable Values	49
Using Variables	49
Numeric Variable Precision	50

Removing Variables	51
4. Control Flow	52
Condition Expressions	52
Conditional Command Execution	53
if Statement	53
switch Statement	54
Loops	55
while Statement	55
for Statement	56
foreach Statement	56
foreach_in_collection Statement	57
Loop Control and Termination	57
continue Statement	58
break Statement	58
5. Working With Procedures	59
Creating Procedures	59
Variable Scope	60
Argument Defaults	61
Variable Numbers of Arguments	62
Using Arrays With Procedures	62
General Considerations for Using Procedures	63
Extending Procedures	63
Using the define_proc_attributes Command	64
define_proc_attributes Command Example	66
Using the parse_proc_arguments Command	66
Considerations for Extending Procedures	68
Displaying the Procedure Body and Arguments	69
6. Collections of Design Objects	70
Why Use Collections?	70
Working With Collections	71
Creating Collections	72
Saving Collections	74
Accessing Collections	74

Contents

Displaying Objects in a Collection	75
Adding Objects to a Collection	76
Removing Objects From a Collection	76
Comparing Collections	77
Extracting Individual Objects From a Collection	78
Iterating Over a Collection	78
Iteratively Adding Objects to a Collection	79
Iteratively Removing Objects From a Collection	79
Working With Attributes	80
Setting and Querying Attribute Values	80
Filtering Collections by Attribute Values	81
Using Filter Expressions	81
Chained Attribute Access	84
User-Defined Attributes	84
Derived User Attributes	85
Using Name-Based Object Specifications	87
Matching Names of Design Objects During Queries	87
Reporting Collections	89
Writing Collection Information to a File	91
7. Using Scripts	93
Using Command Scripts	93
Creating Scripts	94
Using the Output of the write_script Command	94
Running Command Scripts	95
Running Scripts From Within the Tool	95
Running Scripts During Tool Invocation	95
8. A Tcl Script Example	97
rpt_cell Overview	97
rpt_cell Listing and Output Example	98
rpt_cell Details	102
Defining the Procedure	102
Suppressing Warning Messages	103
Examining the Procedure Argument	103

Contents

Initializing Variables	105
Creating and Iterating Over a Collection	106
Collecting the Report Data	106
Formatting the Output	109

9. Command-Line Editor	111
Changing the Settings of the Command-Line Editor	111
Listing Key Mappings	112
Setting the Key Bindings	112
Navigating the Command Line	113
Completing Commands, Variables, and File Names	113
Searching the Command History	114

About This Manual

This manual describes how to use the open source scripting tool, Tcl (tool command language), that has been integrated into Synopsys tools. This manual provides an overview of Tcl, describes its relationship with Synopsys command shells, and explains how to create scripts and procedures.

The audience for *Using Tcl With Synopsys Tools* is designers who are experienced with using Synopsys tools such as Design Compiler and IC Compiler and who have a basic understanding of programming concepts such as data types, control flow, procedures, and scripting.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about Synopsys tools, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following Tcl-based Synopsys products:

- Design Compiler®
- DC Explorer
- Design Vision™
- DFT Compiler and DFTMAX™

- Formality®
- HDL Compiler™
- IC Compiler™
- IC Compiler™ II
- Power Compiler™
- PrimeTime®
- TetraMAX®

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code>
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.

Convention	Description
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

1

Introduction to the Tool Interfaces

This chapter provides the information you need to run a Synopsys Galaxy Platform tool. This chapter consists of the following sections:

- [Tcl and Synopsys Tools](#)
- [Tool Interfaces](#)
- [Starting the Command-Line Interface](#)
- [Using Setup Files](#)
- [Including Tcl Scripts](#)
- [Using Command Log Files](#)
- [Using the Filename Log File](#)
- [Interrupting Commands](#)
- [Controlling Information, Warning, and Error Messages](#)
- [Running Linux Commands Within the Tool](#)
- [Exiting the Tool](#)

Tcl and Synopsys Tools

Tcl is a widely used scripting tool that was developed for controlling and extending applications. Tcl was created by John K. Ousterhout at the University of California, Berkeley, and is distributed as open source software. Tcl is used by many Synopsys command shells as a scripting tool for automating the design processes.

Tcl provides the necessary programming constructs—variables, loops, procedures, and so forth—for creating scripts with Synopsys commands.

Note that it is the scripting language, not the Tcl shell, that is integrated into the Synopsys tools. This aspect of Tcl encompasses how variables, expressions, scripts, control flow, and procedures work, as well as the syntax of commands, including Synopsys commands.

The examples in this book use a mixture of Tcl and Synopsys commands, so when necessary for clarity, a distinction is made between the Tcl and Synopsys commands. Furthermore, the Tcl commands that differ from their base implementation are referred to as Synopsys commands. These commands are `exit`, `history`, `rename`, and `source`. You can refer to the Synopsys man pages for a description of how these commands have been implemented.

If you try to execute the examples in this book, you must do so within a Synopsys command shell because the Tcl shell does not support the Synopsys commands.

Note:

The Synopsys commands are distributed per license agreement for a particular Synopsys tool or product. Because of this, your particular command shell might not support some of the commands used in the examples. Also, some Synopsys shells implement a special mode for handling Tcl commands that you might have to consider. As for the Tcl commands, almost all are supported by the Synopsys command shells.

Most Tcl commands supported by the Synopsys shells use a one-word form. The majority of the Synopsys commands have a multiple-word form in which each word is separated by an underscore, for example, `foreach_in_collection` or `set_host_options`. However, there are also a number of one-word Synopsys commands.

The Tcl commands are referred to as built-in commands by the Synopsys `help` command and as Tcl built-in commands by the Synopsys man pages. To see the list of Tcl built-in commands, enter the following command:

```
prompt> help Builtins
```

Tool Interfaces

Most Synopsys tools offer two interfaces: a command-line interface (or shell) and a graphical user interface (GUI).

- The command-line interface is a text-only environment in which you enter the commands at the command-line prompt.
- The GUI provides tools for visualizing design data and analyzing results.

The command-line interface is based on Tcl. The tool command set includes both Tcl built-in commands, which provide capabilities similar to Linux command shells, including variables, conditional execution of commands, and control flow commands, and application commands, which are command extensions needed to implement specific tool functionality.

You can execute commands within the tool in the following ways:

- By entering single commands interactively in the shell
- By running one or more command scripts, which are text files of commands
- By typing single commands interactively on the console command line in the GUI.

You can use this approach to supplement the subset of application commands available through the menu interface. For more information about the Design Compiler GUI, Design Vision, see the *Design Vision User Guide* and Design Vision online Help. For more information about the IC Compiler GUI, see the IC Compiler online Help.

Starting the Command-Line Interface

Synopsys Tcl-based tools operate in the Linux environment. Before starting the tool, make sure the path to the bin directory is included in your `PATH` environment variable.

To start the command-line interface for a tool, enter the name of its shell command at the Linux system prompt. For example, to invoke Design Compiler,

```
linux% dc_shell
```

Note:

When you invoke a tool, it automatically executes the commands in the setup files. Setup files can perform basic tasks, such as initializing options and variables, declaring design libraries, and setting GUI options. For more details, see [Using Setup Files](#).

When you start the command-line interface, the tool's shell prompt appears in the Linux shell. If you need to use the GUI after starting the command-line interface, enter the `gui_start` command at the shell prompt.

You can include other options on the command line when you start the command-line interface. Each tool has its own set of available options. To see the list of available options, use the `-help` option. For example,

```
linux% dc_shell -help
```

For a complete list of startup options, see the tool command man page (which you can view inside the tool), or see the tool documentation.

At startup, the tool performs the following tasks:

1. Creates a command log file
2. Reads and executes the setup files

3. Executes any script files or commands specified by using the `-f` and `-x` options, respectively, on the command line
4. Displays the program header and shell prompt in the shell in which you started the tool
The program header lists all tool features for which your site is licensed.

Using Setup Files

Setup files can contain commands that perform basic tasks, such as initializing options and variables, declaring libraries, and setting GUI options. Linux environmental variables (such as `$SYNOPSYS`) are accessed by using the `getenv` command or the `$env()` Tcl array variable.

When you invoke a tool, it reads the setup files from three directories, which are searched in the following order:

1. The Synopsys root directory.

These system-wide setup files reside in the `$SYNOPSYS/admin/setup` directory. They contain general tool setup information for all users at your site. You should not edit these files.

2. Your home directory.

This user-defined setup file contains your preferences for your working environment.

3. The directory from which you start the tool.

This design-specific setup file contains project-specific or design-specific variables.

If the setup files share commands or variables, values in the most recently read setup file override values in previously read files. For example, the working directory's settings override any default settings in your home directory or the Synopsys root directory.

[Table 1](#) shows the setup file names for several Synopsys tools.

Table 1 Setup File Names for Synopsys Tcl-Based Tools

Tool name	Setup file name(s)
Design Compiler, Design Compiler NXT	<code>.synopsys_dc.setup</code>
ESP	<code>.synopsys_esp.setup</code>
Fusion Compiler	<code>.synopsys_fc.setup</code>
IC Compiler	<code>.synopsys_dc.setup</code> , then <code>.synopsys_icc.tcl</code>

Table 1 *Setup File Names for Synopsys Tcl-Based Tools (Continued)*

Tool name	Setup file name(s)
IC Compiler II	.synopsys_icc2.setup
PrimeTime	.synopsys_pt.setup
Formality	.synopsys_fm.setup
Library Compiler	.synopsys_lc.setup
TetraMAX, TestMAX ATPG	.tmaxrc ¹

If you want to prevent the tool from reading the setup files, use the `-no_init` option when you invoke the tool.

The setup file in the Synopsys root directory must use the following subset of Tcl built-in commands:

<code>alias</code>	<code>if</code>	<code>setenv</code>
<code>annotate</code>	<code>info</code>	<code>sh</code>
<code>define_name_rules</code>	<code>list</code>	<code>source</code>
<code>define_design_lib</code>	<code>quit</code>	<code>string</code>
<code>exit</code>	<code>redirect</code>	
<code>getenv</code>	<code>set</code>	
<code>group_variable</code>	<code>set_svf</code>	

The setup files in your home and project directories can include any tool commands, as well as Tcl procedures.

Note:

Certain setup variables must be set before you start the tool. Changing these variables after you have started the tool has no effect. An example of such a variable is the `shell.common.enable_line_editing` application option, that enables the command-line editor.

1. See “Setup Command Files” in TetraMAX Online Help for more details.

Including Tcl Scripts

You can use Tcl scripts to accomplish routine, repetitive, or complex tasks. To run a script from the command line, enter the `source file_name` command, where `file_name` is the name of the script file.

You can create a script file by placing a sequence of commands in a text file. Any tool command can be executed within a script file.

In Tcl, a “#” at the beginning of a line denotes a comment. For example,

```
# This is a comment
```

For more information about writing scripts and script files, see [Using Scripts](#).

You can also run scripts when you start the tool. For more information about using startup scripts, see [Starting the Command-Line Interface](#).

Using Command Log Files

The command log file records the commands processed by the tool, including the setup file commands and variable assignments. By default, the tools write the command log to a file in the directory from which you invoked the tool.

[Table 2](#) shows the setup file names for several Synopsys tools.

Table 2 Command Log File Names for Synopsys Tcl-Based Tools

Tool name	Command log file name
Design Compiler, Design Compiler NXT	command.log
ESP	esp_command.log
Fusion Compiler	fc_command.log
IC Compiler	command.log
IC Compiler II	icc2_command.log
PrimeTime	pt_shell_command.log
Formality	fm_shell_command.log
Library Compiler	lc_shell_command.log
TetraMAX	log_file

You can change the name of the command log file by setting the `sh_command_log_file` variable in your setup file. You should make any changes to this variable before you start the tool. If your user-defined or project-specific setup file does not contain this variable, the tool automatically creates the `command.log` file.

Each tool session overwrites the command log file. To save a command log file, move it or rename it. You can use the command log file to

- Produce a script for a particular implementation strategy
- Record the implementation process
- Document any problems you are having

If you do not want the tool to create the `command.log` file, you can do one of the following:

- Set the `sh_command_log_file` variable to `/dev/null`.
- Use the `-no_log` option when you start the tool.

Using the Filename Log File

The Design Compiler and IC Compiler tools generate a filename log file that contains a list of the files read by the tool. You can use this file to identify data files needed to reproduce an error if the tool terminates abnormally.

By default, the filename log file is named `filename.log` and is written to the directory from which you invoked the tool. You can change the name of the filename log file by setting the `filename_log_file` variable in your setup file. You should make any changes to this variable before you start the tool. If you started the tool with the `-no_log` option, it appends the process ID of the application and date stamp to the name of the filename log file.

You can have the tool append a process ID to all filename log files by including the following settings in your setup file:

```
set _pid [pid]
set filename_log_file filename.log$_pid
```

By default, the tool automatically removes the filename log file when you exit. To save the filename log file, set the `exit_delete_filename_log_file` variable to `false`.

Other tools do not have this feature.

Interrupting Commands

If you enter the wrong options for a command or enter the wrong command, you can interrupt command processing and remain in the shell. To interrupt a command, press Ctrl+C.

The time it takes for the command to respond to an interrupt (to stop what it is doing and continue with the next command) depends on the size of the design and the command being interrupted.

Some commands cannot be interrupted. To stop such commands, you must terminate the shell at the system level by using operating system commands such as the `kill` command.

When you use Ctrl+C, keep the following points in mind:

- If a script file is being processed and you interrupt one of its commands, the script processing is interrupted and no further commands in the script file are processed.
- In general, when you terminate a command, no data is saved.

For tool-specific details on Ctrl+C handling, see the tool documentation.

- If you press Ctrl+C three times before a command responds to your interrupt, the shell itself is interrupted and exits with this message:

```
Information: Process terminated by interrupt.
```

This behavior has a few exceptions, which are documented in the man pages for the applicable commands.

Controlling Information, Warning, and Error Messages

By default, the tools display informational, warning, and error messages. To disable printing of informational or warning messages, use the `suppress_message` command with the list of message IDs you want to suppress.

For example, to suppress the CMD-029 message, use the following command:

```
prompt> suppress_message CMD-029
```

To display the currently suppressed message IDs, use the `print_suppressed_messages` command.

To reenable printing of the suppressed messages, use the `unsuppress_message` command.

PrimeTime tool does not support `suppress_errors` warning messages.

To disable printing of error messages, use the `suppress_errors` variable with a list of error message IDs for which you want messages to be suppressed.

Running Linux Commands Within the Tool

The tools support some common Linux commands, as listed in [Table 3](#).

Table 3 Common Tasks and Their System Commands

To do this	Use this
List the current working directory.	<code>pwd</code>
Change the working directory to a specified directory or, if no directory is specified, to your home directory.	<code>cd directory</code>
List the specified files, or, if no arguments are specified, list all files in the working directory.	<code>ls directory_list</code>
Search for a file, using the search path defined by the <code>search_path</code> variable.	<code>which filename</code>
Return the value of an environment variable.	<code>getenv name</code> or <code>set env(name)</code>
Set the value of an environment variable. Any changes to environment variables apply only to the current process and to any child processes launched by the current process.	<code>setenv name value</code> or <code>set env(name) value</code>
Display the value of an environment variable (or all if no variable specified).	<code>printenv variable_name</code>
Execute an operating system command. This Tcl built-in command has some limitations. For example, no file name expansion is performed.	<code>exec command</code>
Execute an operating system command. Unlike <code>exec</code> , this command performs file name expansion.	<code>sh command</code>

Although you can use the `sh` command or `exec` command to execute operating system commands, it is strongly recommended that you use native Tcl functions or procedures. After you have loaded designs and libraries, the tool process might be quite large. In such cases, using the `sh` or `exec` commands might be quite slow and on some operating systems, might fail altogether due to insufficient virtual memory. You can use Tcl built-in commands to avoid this problem.

For example, to remove a file from within the tool under Linux, use the following command:

```
prompt> file delete filename
```

For better performance, replace common Linux commands with the Tcl equivalent listed in [Table 4](#).

Table 4 *Tcl Equivalent of Linux Commands*

Linux command	Tcl equivalent
ls	glob for patterns or various file subcommands
rm	file delete
rm -rf	file delete -force
mv	file rename
date	date
sleep	after

It is often possible to replace common Linux commands with simple Tcl procedures. For example, you can use the following procedure to replace the Linux `touch` command.

Example 1 *Tcl Procedure to Replace Linux touch Command*

```
proc touch {file_name} {
    if {[file exists $file_name]} {
        file mtime $file_name [clock seconds]
    } else {
        set fp [open $file_name "w"]
        close $fp
    }
}
```

You can write similar procedures to replace other external commands.

Exiting the Tool

You can exit the tool at any time and return to the operating system. To exit the tool, use the `exit` or `quit` command.

When you exit the tool, the default exit value is always 0. You can assign an exit code value to the `exit` command, which can be useful when you run the tool within a makefile. For example, to assign an exit code value of 8, use the following command:

```
prompt> exit 8
```

```
Memory usage for main task 26 Mbytes.  
Memory usage for this session 26 Mbytes.  
CPU usage for this session 2 seconds ( 0.00 hours ).
```

```
Thank you ...  
% echo $status  
8
```

When you exit the tool by using the `exit` or `quit` command, it does not save the open designs. You must explicitly save the designs before exiting the tool. For information about saving the design, see the tool documentation.

2

Commands

The command set in Synopsys Tcl-based tools includes both Tcl built-in commands and application commands. This chapter describes the command syntax and how to use some basic commands.

This chapter contains the following sections:

- [Command-Line Editor](#)
- [Application Command Syntax](#)
- [Outputting Data to the Screen](#)
- [Command Parsing](#)
- [Redirecting and Appending Command Output](#)
- [Command Status](#)
- [Listing and Rerunning Previously Entered Commands](#)
- [Getting Help on Commands](#)
- [Tcl Limitations Within the Command-Line Interface](#)
- [Basic Tcl Commands](#)

Command-Line Editor

The command-line editor allows you to work interactively with the tool by using key sequences, much as you would work in a Linux shell. The command-line editor is enabled by default. To disable this feature, set the `shell.common.enable_line_editing` application option to `false`. For more information, see [Command-Line Editor](#).

Application Command Syntax

The syntax for an application command is

```
command_name [command_argument] [-option [argument]]
```

command_name

The name of the command.

command_argument

Some commands require values or arguments. Arguments that do not begin with a hyphen (-) are called positional arguments. They must be entered in a specific order relative to each other.

-option

Arguments that begin with a hyphen (-) are called options. Options modify the command behavior and pass information to the tool. Options can be entered in any order and can be intermingled with positional arguments.

argument

The argument to the option. Some options require values or arguments.

Note:

Command and option names are case-sensitive, as are other values, such as file names, design object names, and strings.

Arguments and options can be required or optional. If you omit a required argument or option, the tool issues an error message and a usage statement.

The following example shows an application command with a command argument, but no options:

```
prompt> read_verilog example.v
```

Special Characters

The characters listed in [Table 5](#) have special meaning in certain contexts.

Table 5 *Tcl Special Characters*

Character	Meaning
\$	References a variable.
()	Used for grouping expressions.
[]	Denotes a nested command.
\	Used for escape quoting and character substitution.
" "	Denotes weak quoting. Nested commands and variable substitutions still occur.

Table 5 Tcl Special Characters (Continued)

Character	Meaning
{ }	Denotes rigid quoting. No substitutions are allowed.
*	Wildcard character. Matches zero or more characters.
?	Wildcard character. Matches one character.
;	Ends a command. (Needed only when you place more than one command on a line.)
#	Begins a comment.

Wildcard Character

Tcl has two wildcard characters, the asterisk (*) and the question mark (?). The * wildcard character matches zero or more characters in a name. For example, u* indicates all object names that begin with the letter u, and u*z indicates all object names that begin with the letter u and end in the letter z. The ? wildcard character matches a single character in a name. For example, u? indicates all object names exactly two characters in length that begin with the letter u.

Creating Comments

Comment lines are created by placing a pound sign (#) as the first nonblank character of a line. You can create inline comments by placing a semicolon between a command and the pound sign. For example,

```
echo abc ;# this is an inline comment
```

When the command continuation character (\) is placed at the end of a commented command line, the subsequent line is also treated as a comment. In the following example, none of the set commands are executed:

```
# set CLK_NAME Sysclk; set CLK_PERIOD 10; \  
set INPUT_DELAY 2
```

Data Types

You can use the following data types, which are described in the sections that follow:

- [Strings](#)
- [Lists](#)
- [Arrays](#)

Note:

Synopsys tools also support a collection data type, which is described in [Creating Collections](#).

Strings

A string is a sequence of characters. Tcl treats command arguments as strings and returns command results as strings. The following are string examples:

```
sysclk  
"FF3 FF4 FF5"  
{FF6 FF7}
```

To include special characters, such as space, backslash, or new line, in a string, you must use quoting to disable the interpretation of the special characters.

The Tcl `string` command performs many common string operations. The syntax is

```
string option arg ...
```

The arguments are as follows:

option

Specifies an option for the `string` command.

arg ...

Specifies the argument or arguments for the `string` command.

For example, to compare two strings, use the `compare` option as follows:

```
string compare string1 string2
```

To convert a string to all uppercase characters, use the `toupper` option as follows:

```
string toupper string
```

[Table 6](#) lists Tcl commands you can use with strings. For more information about these commands, see the Synopsys man pages.

Table 6 *Tcl Commands to Use With Strings*

Command	Description
<code>format</code>	Formats a string.
<code>regexp</code>	Searches for a regular expression within a string.
<code>regsub</code>	Performs substitutions based on a regular expression.
<code>scan</code>	Assigns fields in the string to variables.

Table 6 *Tcl Commands to Use With Strings (Continued)*

Command	Description
<code>string</code>	Provides a set of string manipulation functions.
<code>subst</code>	Performs substitutions.

Lists

A list is an ordered group of elements; each element can be a string or another list. You use lists to group items such as a set of cell instance pins or a set of report file names. You can then manipulate the grouping as a single entity.

You can create a simple list by enclosing the list elements in double quotation marks (") or braces ({}), or by using the Tcl `list` command. You must delimit list elements with spaces—do not use commas.

For example, you could create a list of cell instance D-input pins, I1/FF3/D, I1/FF4/D, and I1/FF5/D, in one of the following ways:

```
set D_pins "I1/FF3/D I1/FF4/D I1/FF5/D"
set D_pins {I1/FF3/D I1/FF4/D I1/FF5/D}
set D_pins [list I1/FF3/D I1/FF4/D I1/FF5/D]
```

You use the `list` command to create a compound (nested) list. For example, the following command creates a list that contains three elements, each of which is also a list:

```
set compound_list [list {x y} {1 2.5 3.75 4} {red green blue}]
```

Because braces prevent substitutions, you must use double quotation marks or the `list` command to create a list if the list elements include nested commands or variable substitution.

For example, if variable `a` is set to 5, the following commands generate very different results:

```
prompt> set a 5
5

prompt> set b {c d $a [list $a z]}
c d $a [list $a z]

prompt> set b [list c d $a [list $a z]]
c d 5 {5 z}
```

To access a specific element in a simple or compound list, you use the `lindex` command. For example, the following commands print out the first element of the `D_pins` list and the second element of the `compound_list` list:

```
prompt> lindex $D_pins 0
I1/FF3/D

prompt> lindex $compound_list 1
1 2.5 3.75 4
```

Note that `lindex` is zero based.

[Table 7](#) lists Tcl commands you can use with lists. For more information about these commands, see the Synopsys man pages.

Table 7 *Tcl Commands to Use With Lists*

Command	Task
<code>concat</code>	Concatenates lists and returns a new list.
<code>join</code>	Joins elements of a list into a string.
<code>lappend</code>	Appends elements to a list.
<code>lindex</code>	Returns a specific element from a list.
<code>linsert</code>	Inserts elements into a list.
<code>list</code>	Returns a list formed from its arguments.
<code>llength</code>	Returns the number of elements in a list.
<code>lminus</code>	Removes one or more named elements from a list and returns a new list.
<code>lrange</code>	Extracts elements from a list.
<code>lreplace</code>	Replaces a specified range of elements in a list.
<code>lsearch</code>	Searches a list for a regular expression.
<code>lsort</code>	Sorts a list.
<code>split</code>	Splits a string into a list.

Arrays

Tcl supports associative arrays. This type of array uses arbitrary strings, which can include numbers, as its indexes. The associative array is composed of a group of elements where

each element is a variable with its own name and value. To reference an array element, you use the following form:

```
array_name (element_name)
```

For example, you can create an array of report file name extensions as follows:

```
prompt> set vio_rpt_ext(ir_drop) .volt
.volt
prompt> set vio_rpt_ext(curr_dens) .em
.em
prompt> set vio_rpt_ext(curr) .current
.current
```

The first `set` command creates the `vio_rpt_ext` array and sets its `ir_drop` element to `.volt`. The subsequent commands create new array elements and assign values to them. [Table 8](#) illustrates how the `vio_rpt_ext` array is organized.

Table 8 *Structure of `vio_rpt_ext` Array*

Element names	Element values
<code>ir_drop</code>	<code>.volt</code>
<code>curr_dens</code>	<code>.em</code>
<code>curr</code>	<code>.current</code>

The following example prints out the `curr_dens` element:

```
prompt> echo $vio_rpt_ext(curr_dens)
.em
```

You can use the `array` command, along with one of its options, to get information about the elements of an array. The following commands use the `size` and `names` options to print the size and element names of the `vio_rpt_ext` array.

```
prompt> array size vio_rpt_ext
3
prompt> array names vio_rpt_ext
curr curr_dens ir_drop
```

For more information about array usage, see the `array` man page.

Operators

The Tcl language does not directly provide operators (such as arithmetic, and string and list operators), but Tcl built-ins such as the `expr` command do support operators within expressions.

For example,

```
prompt> set delay [expr .5 * $base_delay]
```

Use the `expr` command to evaluate an expression.

For example, if you want to multiply the value of a variable named `p` by 12 and place the result into a variable named `a`, enter the following commands:

```
prompt> set p 5
5
prompt> set a [expr (12*$p)]
60
```

The following command does not perform the desired multiplication:

```
prompt> set a (12 * $p)
```

Where possible, expression operands are interpreted as integers. Integer values can be decimal, octal, or hexadecimal. Operands not in an integer format are treated as floating-point numbers, if possible. For more information, see [Numeric Variable Precision](#). Operands can also be one of the mathematical functions supported by Tcl.

Note:

The `expr` command is the simplest way to evaluate an expression. You also find expressions in other commands, such as the control flow `if` command. The rules for evaluating expressions are the same whether you use the `expr` command or use the expression within the conditional statement of a control flow command. For more information, see [Control Flow](#).

[Table 9](#) lists the Tcl operators in order of precedence. The operators at the top of the table have precedence over operators lower in the table.

Table 9 *Tcl Operators*

Syntax	Description	Operand types
-a	Negative of a	int, real
!a	Logical NOT: 1 if a is zero, 0 otherwise	int, real
~a	Bitwise complement of a	int

Table 9 Tcl Operators (Continued)

Syntax	Description	Operand types
a*b	Multiply a and b	int, real
a/b	Divide a by b	int, real
a%b	Remainder after dividing a by b	int
a+b	Add a and b	int, real
a-b	Subtract b from a	int, real
a<<b	Left-shift a by b bits	int
a>>b	Right-shift a by b bits	int
a<b	1 if a is less than b, 0 otherwise	int, real, string
a>b	1 if a is greater than b, 0 otherwise	int, real, string
a<=b	1 if a is less than or equal to b, 0 otherwise	int, real, string
a>=b	1 if a is greater than or equal to b, 0 otherwise	int, real, string
a==b	1 if a is equal to b, 0 otherwise	int, real, string
a!=b	1 if a is not equal to b, 0 otherwise	int, real, string
a&b	Bitwise AND of a and b	int
a^b	Bitwise exclusive OR of a and b	int
a b	Bitwise OR of a and b	int
a&&b	Logical AND of a and b	int, real
a b	Logical OR of a and b	int, real
a?b:c	If a is nonzero, then b, else c	a: int, real b, c: int, real, string

Abbreviating Command and Option Names

You can abbreviate application command names and options to the shortest unambiguous (unique) string. For example, you can abbreviate the `get_attribute` command to `get_attr` or the `create_clock` command's `-period` option to `-p`. However, you cannot abbreviate most built-in commands.

By default, you can use command abbreviations either interactively or in scripts. You can control whether command abbreviation is enabled by using the `sh_command_abbrev_mode` variable. The valid values are `Anywhere`, `Command-Line-Only`, and `None`. The default is `Anywhere`. To specify that command abbreviation is enabled only interactively, set the variable to `Command-Line-Only`. To disable command abbreviations completely, set the variable to `None`.

To determine the current value of the `sh_command_abbrev_mode` variable, enter

```
prompt> get_app_var sh_command_abbrev_mode
```

Command abbreviation is meant as an interactive convenience. Do not use command or option abbreviation in script files because script files are susceptible to command changes in subsequent versions of the application. Such changes can cause abbreviations to become ambiguous.

If you enter an ambiguous command, the tool attempts to help you find the correct command.

For example, the `set_min_` command as entered here is ambiguous:

```
prompt> set_min_  
Error: ambiguous command 'set_min_' matched 4 commands:  
(set_min_capacitance, set_min_delay, set_min_library ...) (CMD-006)
```

The tool lists up to three of the ambiguous commands in its error message. To list the commands that match the ambiguous abbreviation, use the `help` command with a wildcard pattern. For example,

```
prompt> help set_min_*  
set_min_capacitance # set min capacitance  
set_min_delay       # set min delay  
set_min_library     # set min library  
set_min_pulse_width # set min pulse width
```

Using Aliases

You can use aliases to create short forms for the commands you commonly use. When you use aliases, keep the following points in mind:

- Alias names can include letters, digits, underscores, and punctuation marks, but they cannot begin with a digit.
- Alias names are case-sensitive.
- You cannot use an existing command name as an alias name; however, aliases can refer to other aliases.
- The command-line interface recognizes aliases only when they are the first word of a command.
- An alias definition takes effect immediately but lasts only until you exit the session. To save commonly used alias definitions, store them in the setup file.

To create an alias, use the `alias` command. To create an alias for a multiword command, you must format the command as a Tcl list (enclose it in curly braces or quotation marks).

For example, the following command defines *rt100* as an alias for the `report_timing -max_paths 100` command:

```
prompt> alias rt100 {report_timing -max_paths 100}
```

To list all aliases defined in the current session, use the `alias` command without an argument.

To remove alias definitions created with the `alias` command, use the `unalias` command. To remove all aliases, use the `-all` option; otherwise, specify the aliases you want to remove.

For example, to remove all aliases beginning with `f*` and the `rt100` alias, enter

```
prompt> unalias f* rt100
```

Multiple Line Commands and Multiple Commands per Line

If you enter a long command with many options and arguments, you can split it across more than one line by using the continuation character, the backslash (`\`). There is no limit to the number of characters in a command line.

Type only one command on a single line; if you want to put more than one command on a line, separate the commands with a semicolon.

Outputting Data to the Screen

The `echo` and `puts` commands allow you to output data to the screen. The `echo` command prints its argument to the console window.

Note:

The Synopsys implementation of the `echo` command varies from the Tcl implementation. For usage information about the `echo` command, see the Synopsys man pages.

The `puts` command, when used in its simplest form, prints its argument to the standard output. Note that the console window might not be the same as the standard output. The console window is an integral component of the Synopsys tool you are running, and the standard output is, by default, the operating system command shell from which you invoked your Synopsys tool.

The syntax for the `echo` command is

```
echo [-n] argument
```


The arguments are as follows:

`-n`

Suppresses output of the new-line character output.

argument

The item to output.

The following example prints a line of text and a new line to the console window:

```
prompt> echo "Have a good day."  
Have a good day.
```

The syntax for the `puts` command is

```
puts [-nonewline file_id] arg
```

The arguments are as follows:

`-nonewline`

Suppresses output of the new-line character.

file_id

Specifies the file ID of the channel to which to send the output. If not specified, the output is sent to the standard output.

arg

The item to output.

The following example shows how to use `puts` in its simplest form:

```
prompt> puts "Have a good day."  
Have a good day.
```

Command Parsing

A Synopsys command shell parses commands (Tcl and Synopsys) and makes substitutions in a single pass from left to right. At most, a single substitution occurs for each character. The result of one substitution is not scanned for further substitutions.

Substitution

The substitution types are:

- Command substitution

You can use the result of a command in another command (nested commands) by enclosing the nested command in square brackets (`[]`).

For example,

```
prompt> set a [expr 24 * 2]
```

You can use a nested command as a conditional statement in a control structure, as an argument to a procedure, or as the value to which a variable is set. Tcl imposes a depth limit of 1,000 for command nesting.

Synopsys tools make one exception to the use of square brackets to indicate command nesting—you can use square brackets to indicate bus references. Synopsys tools accept a string, such as `data[63]`, as a name rather than as the word `data` followed by the result of the command `63`.

- Variable substitution

You can use variable values in commands by using the dollar sign character (`$`) to reference the value of the variable. (For more information about Tcl variables, see [Variables](#).)

For example,

```
prompt> set a 24
24
prompt> set b [expr $a * 2]
48
```

- Backslash (`\`) substitution

You use backslash substitution to insert special characters, such as a new line, into text. For example,

```
prompt> echo "This is line 1.\nThis is line 2."
This is line 1.
This is line 2.
```

You can also use backslash substitution to disable special characters when weak quoting is used. See [Quoting](#).

Quoting

You use quoting to disable the interpretation of special characters (for example, [], \$, and ;). You disable command substitution and variable substitution by enclosing the arguments in braces ({}); you disable word and line separators by enclosing the arguments in double quotation marks (").

Braces specify rigid quoting. Rigid quoting disables all substitution, so that the characters between the braces are treated literally. For example,

```
prompt> set a 5; set b 10
10
prompt> echo {[expr $b - $a]} evaluates to [expr $b - $a]
[expr $b - $a] evaluates to 5
```

Double quotation marks specify weak quoting. Weak quoting disables word and line separators while allowing command, variable, and backslash substitution. For example,

```
prompt> set A 10; set B 4
4
prompt> echo "A is $A; B is $B.\nNet is [expr $A - $B]."
```

A is 10; B is 4.
Net is 6.

Special Characters

[Table 10](#) lists the characters that have special meaning in Tcl. If you do not want these characters treated specially, you can precede the special characters with a backslash (\).

For example,

```
prompt> set gp 1000; set ex 750
750
prompt> echo "Net is: \${expr $gp - $ex}"
Net is: $250
```

Table 10 *Tcl Special Characters*

Character	Meaning
\$	Used to access the value of a variable.
()	Used to group expressions.
[]	Denotes a nested command. (For an exception, see Substitution .)
\	Used for escape quoting and as a line continuation character.

Table 10 *Tcl Special Characters (Continued)*

Character	Meaning
""	Denotes weak quoting. Nested commands and variable substitutions still occur.
{ }	Denotes rigid quoting. There are no substitutions.
;	Ends a command.
#	Begins a comment.

Bus Notation

Normally, Tcl interprets anything in square brackets as a command. This includes the index values in square brackets used for bus notation. However, as a convenience, the tool allows common bus notations to be used without quoting or escaping the square brackets.

Table 11 *Supported Bus Notations for Unquoted Names*

Bus notation type	Example
Integer value	prompt> get_ports A[3] {A[3]}
Hexadecimal value	prompt> get_ports B[0xff] {B[0xff]}
Octal value	prompt> get_ports C[040] {C[040]}
Colon-separated value range	prompt> get_cells my_multibit_reg[3:0] {my_multibit_reg[3:0]}
Asterisk (*) wildcard	prompt> get_nets my_bus[*] {my_bus[1] my_bus[0]}

Bus notations not in this list must be quoted or escaped. For example,

```
prompt> get_cells REG_ARRAY[3_2] ;# '3_2' interpreted as command
Error: unknown command '3_2' (CMD-005)
```

```
prompt> get_cells REG_ARRAY\[3_2\] ;# escaped
{REG_ARRAY[3_2]}
```

```
prompt> get_cells {REG_ARRAY[3_2]} ;# quoted  
{REG_ARRAY[3_2]}
```

For details on how this feature is implemented, see [SolvNetPlus article 000016885](#), “How Do Synopsys Tcl Tools Handle Unquoted Bus Notation?”

Redirecting and Appending Command Output

If you run scripts overnight, you cannot see warnings or error messages echoed to the command window while your scripts are running. You can direct the output of a command, procedure, or script to a specified file in two ways:

- By using the traditional Linux redirection operators (> and >>)
- By using the `redirect` command

Using the Redirection Operators

You can use the Linux redirection operators (> and >>) in the following ways:

- Divert command output to a file by using the redirection operator (>).
- Append command output to a file by using the append operator (>>).

Note:

You cannot use the Linux style redirection operators with built-in (non-Synopsys) Tcl commands. Always use the `redirect` command when using such commands.

The Tcl built-in command `puts` does not respond to redirection of any kind. Instead, use the `echo` command, which responds to redirection.

Because Tcl is a string-oriented language, traditional operators usually have no special meaning unless a particular command (such as `expr`) imposes some meaning. Application commands respond to > and >> but, unlike Linux, the command-line interface treats > and >> as arguments to a command. Therefore, you must use white space to separate these arguments from the command and the redirected file name. For example,

```
prompt> echo $my_variable >> file.out; # Right
```

```
prompt> echo $my_variable>>file.out; # Wrong!
```

Using the redirect Command

You can direct command output to a file by using the `redirect` command. The `redirect` command performs the same function as the traditional Linux redirection operators (>

and >>); however, the `redirect` command is more flexible. For example, you can direct command output to the standard output device as well as a file by using the `-tee` option.

Also, the Linux redirection operators are not part of the Tcl language and cannot be used with built-in commands. You must use the `redirect` command with built-in commands.

Command Status

Every application command returns a value, either a status code or design-specific information.

Command status codes in the command-line interface are

- 1 for successful completion
- 0 or { } (null list) for unsuccessful execution

Successful Completion Example

The command status value returned for the `alias` command is 1, indicating successful command completion.

```
prompt> alias zero_del "set_max_delay 0.0 all_outputs()"
prompt> zero_del
1
```

Unsuccessful Execution Examples

If a command cannot be executed properly, its return value is an error status code. The error status value is 0 for most commands, a null list ({}) for commands that return a list, and an empty string for commands that return a collection.

```
prompt> set_driving_cell -lib_cell IV {I1}
Error: Cannot find the specified driving cell in memory. (UID-993)
0
```

Listing and Rerunning Previously Entered Commands

You can use the `history` command to list and execute previously entered commands. If you use the `history` command without options, a list of executed commands is printed; by default, 20 commands are listed. The list of commands is printed as a formatted string that shows the event number for each command.

You use the `info` option of the `history` command to list a specific number of previously entered commands. For example, the following command lists the last five executed commands:

```
prompt> history info 5
```

You use the `redo` option of the `history` command to reexecute a specific command. You can specify the command to reexecute by its event number or by a relative event number.

The following command executes the command whose event number is 54:

```
prompt> history redo 54
```

The following command reexecutes the second-to-the-last command:

```
prompt> history redo -2
```

If you do not specify an event number, the last command entered is reexecuted.

As a shortcut, you can also use the exclamation point operator (!) for reexecuting commands. For example, to reexecute the last command, enter

```
prompt> !!
```

To reexecute the command whose event number is 6, enter

```
prompt> !6
```

Note:

The Synopsys implementation of `history` varies from the Tcl implementation. For usage information about the `history` command, see the Synopsys man pages.

Getting Help on Commands

To get help about a command or variable, use the `help` or `man` command. Additionally, you can display a command's options and arguments by using the `-help` option. For example,

```
prompt> create_clock -help
Usage: create_clock # create clock
[-name clock_name] (name for the clock)
[-period period_value] (period of the clock: Value >= 0)
[-waveform edge_list] (alternating rise, fall times for 1 period)
[-add] (add to the existing clock in port_pin_list)
[source_objects] (list of ports and/or pins)
```

Note:

To distinguish between Tcl and Synopsys commands, the Synopsys `help` and `man` commands categorize Tcl commands as built-in commands and Tcl built-in commands, respectively.

Using the help Command

The syntax for the `help` command is:

```
help -verbose pattern
```

where the `-verbose` and `pattern` arguments, as follows, are optional:

`-verbose`

Displays a short description of the command arguments.

`pattern`

Specifies a command pattern to match.

Use the `help` command to get help on one or more commands. Use the `-verbose` option to see a list of the command's arguments and a brief description of each argument.

If you use the `help` command without arguments, a list of all commands arranged by command group (for example, Procedures, Builtins, and Default) is displayed.

Specify a command pattern to view help on one or more commands. For example, the following command shows help for all commands starting with `for`:

```
prompt> help for*
```

You can get a list of all commands for a particular command group by entering a command group name as the argument to the `help` command. For example,

```
prompt> help Procedures
```

Using the man Command

To get help from the Synopsys man pages, use the `man` command, as shown:

```
prompt> man query_objects
```

The man pages provide detailed information about commands and variables.

The syntax for the `man` command is

```
man topic
```


The *topic* argument can be a command or a topic. For example, you can get information about a specific command, such as `query_objects`, or you can get information about a topic, such as attributes.

The man pages are also available on SolvNet.

Tcl Limitations Within the Command-Line Interface

Generally, the command-line interface implements all the Tcl built-in commands. However, the command-line interface adds semantics to some Tcl built-in commands and imposes restrictions on some elements of the language. These differences are as follows:

- The Tcl `rename` command is not supported.
- The Tcl `load` command is not supported.
- You cannot create a command called `unknown`.
- The Tcl `source` command has additional options: `-echo` and `-verbose`.
- The `history` command has several options and forms not supported by Tcl: the `-h` and `-r` options and the `history #` form.
- Because the command-line interface processes words that look like bus (array) notation (words that have square brackets, such as `a[0]`), Tcl does not try to execute the nested command 0. Without this processing, you would need to rigidly quote such array references, as in `{a[0]}`.

Using braces (`{ }`) around all control structures, procedure argument lists, and so on is recommended practice. Because of this extension, however, braces are not only recommended but required. For example, the following code is valid Tcl but will be misinterpreted:

```
if ![expr $a > 2]
{echo "hello world"}
```

Instead, quote the if condition as follows:

```
if {![expr $a > 2]}
{echo "hello world"}
```

Basic Tcl Commands

This section provides an overview of Tcl commands you can use when working with files. You use these commands to work with directories, retrieve information about files, and read from and write to files.

cd and pwd

The `cd` and `pwd` commands are equivalent to the operating system commands with the same name. You use the `cd` command to change the current working directory and the `pwd` command to print the full path name of the current working directory.

file and glob

To retrieve information about a file, use the `file` command. The `file` command has the following syntax:

```
file option argument argument ...
```

Table 12 provides a list of `file` command options.

Table 12 File Command Options

File command and option	Description
<code>file dirname fname</code>	Returns the directory name part of a file name.
<code>file exists fname</code>	Returns 1 if the file name exists, 0 otherwise.
<code>file extension fname</code>	Returns the extension part of a file name.
<code>file isdirectory fname</code>	Returns 1 if the file name is a directory, 0 otherwise.
<code>file isfile fname</code>	Returns 1 if the file name is a file, 0 otherwise.
<code>file readable fname</code>	Returns 1 if the file is readable, 0 otherwise.
<code>file rootname fname</code>	Returns the name part of a file name.
<code>file size fname</code>	Returns the size, in bytes, of a file.
<code>file tail fname</code>	Returns the file name from a file path string.
<code>file writable fname</code>	Returns 1 if the file is writable, 0 otherwise.

To generate a list of file names that match one or more patterns, use the `glob` command. The `glob` command has the following syntax:

```
glob pattern1 pattern2 pattern3 ...
```

The following example generates a list of `.em` and `.volt` files located in the current directory:

```
set flist [glob *.em *.volt]
```

open, close, and flush

You use the `open`, `close`, and `flush` commands to set up file access.

The `open` command syntax is as follows:

```
open fname access_mode
```

The `access_mode` argument specifies how you want the file opened; the default access mode is read-only. Typical access modes include read-only, write only, read and write, and append. For a complete list of all access modes, see the man page for the `open` command. [Table 13](#) lists some commonly used access modes.

Table 13 Commonly Used Access Modes

Access mode	Description
<code>r</code>	Opens the file for reading only; the file must already exist. This is the default access mode.
<code>r+</code>	Opens the file for reading and writing; the file must already exist.
<code>w</code>	Opens the file for writing only. If the file exists, truncates it. If the file does not exist, creates it.
<code>w+</code>	Opens the file for reading and writing. If the file exists, truncates it. If the file does not exist, creates it.
<code>a</code>	Opens the file for writing only; new data is appended to the file. The file must already exist.
<code>a+</code>	Opens the file for reading and writing. If the file does not exist, creates it. New data is appended to the file.

The `open` command returns a string (a file ID) that is used to identify the file for further interaction with it.

You use the `close` command to close a file; it has the following syntax:

```
close $fid
```

The `$fid` argument is the file ID of the file that was obtained from an `open` command.

The following example demonstrates the use of the `open` and `close` commands:

```
set f [open VDD.em w+]
close $f
```

You use the `flush` command to force buffered output to be written to a file. Data written to a file does not always immediately appear in the file when a buffered output scheme is

used. Instead, the data is queued in memory by the system and is written to the file later; the `flush` command overrides this behavior.

The `flush` command has the following syntax:

```
flush $fid
```

gets and puts

You use the `gets` command to read a single line from a file and the `puts` commands to write a single line to a file.

The `gets` command has the following syntax:

```
gets $fid var
```

The `$fid` argument is the file ID of the file that was obtained from an `open` command; the `var` argument is the variable that is to receive the line of data.

After the line is read, the file is positioned to its next line. The `gets` command returns a count of the number of characters actually read. If no characters are read, `gets` returns -1 and places an empty string into `var`.

The `puts` command has the following syntax:

```
puts $fid var
```

The `$fid` argument is the file ID of the file that was obtained from an `open` command; the `var` argument contains the data that is to be written. The `puts` command adds a new-line character to the data before it is outputted.

If you leave out the file ID, the data is written to the standard output. For more information about this use of the `puts` command, see [Outputting Data to the Screen](#).

The following example demonstrates the use of the `gets` and `puts` commands:

```
# Write out a line of text, then read it back and print it
set fname "mytext.txt"
# Open file, then write to it
set fid [open $fname w+]
puts $fid "This is my line of text."
close $fid
#
# Open file, then read from it
set fid [open $fname r]
set data_in [gets $fid]
close $fid
#
# Print out data read
echo $data_in
```

Nonsequential File Access

By default, the `gets` and `puts` commands access files sequentially. You can use the `seek`, `tell`, and `eof` commands to manage nonsequential file access.

You use the `seek` command to move the *access position* of the file by a specified number of bytes. The access position is the point where the next read or write occurs in the file. By default, the access point is where the last read or write ended.

The simplest form of the `seek` command is

```
seek $fid offset
```

The `$fid` argument is the file ID of the file that was obtained from an `open` command; the `offset` argument is the number of bytes to move the access position.

You use the `tell` command to obtain the current access position of a file.

The basic syntax of the command is

```
tell $fid
```

You use the `eof` command to test whether the access position of a file is at the end of the file. The command returns 1 if true; otherwise, it returns a 0.

3

Variables

This chapter describes the use of variables within the command-line interface. Variables store values that commands use. The value of a variable can be a list of pin names, the estimated load on a port, and the like. When you set a variable to a value, the change takes place immediately, and commands use that variable value. This chapter includes the following sections:

- [Components of a Variable](#)
- [Application Variables](#)
- [Considerations When Using Variables](#)
- [Manipulating Variables](#)

Components of a Variable

In Tcl, each variable has a name and a value. The name is a sequence of characters that describe the variable. Values can be any of the supported data types. The data types are described in [Data Types](#). A valid value can be a file name or a list of file names, a number, or a set of command options and arguments. Variable names are case-sensitive

You can store a list of values in a single variable name. For example, you can find the designs in memory and store the list in a variable.

```
prompt> set active_design_list [query_objects [get_designs *]]
```

Application Variables

The Synopsys Tcl-based tools predefine some variable names that control tool-specific features or provide tool-specific information. These predefined variables are called application variables. You can also define new variables, which are called user-defined variables. For example, the `search_path` variable tells the tool where to search for referenced files.

Tcl also provides a few predefined variables, such as the `env` variable. The `env` variable is an array that contains the environment variable names of the Linux shell in which the Synopsys command shell is running. For more information about arrays, see [Arrays](#).

You can view a list of the environment variables by using the `array` command with its `names` option. For example,

```
prompt> array names env
```

The list that prints out contains element names that correspond to the names of environment variables. To reference the value of an environment variable, use `$env(ENV_VAR_NAME)`. For example, you can view the value of the `HOME` environment variable by entering

```
prompt> echo $env(HOME)
```

You can also use the `getenv` command to view the value of an environment variable. For example,

```
prompt> getenv HOME
```

If you change the value of an `env` element, the change is reflected in the environment variable of the process in which the command shell is running. The `env` element is returned to its previous value after the command shell exits.

To list all the application variables, use the `report_app_var` command.

Considerations When Using Variables

Keep in mind these facts about variables when you use them:

- Variable names can include letters, digits, underscores, and punctuation marks.
- Variable names are case-sensitive.
- Variables defined within a procedure are local to that procedure.
- Variables are not saved in the design database. When a session ends, the variables assigned in that session are lost.
- Type conversion is automatic.
- An unquoted string is considered to be a string value.
- You must put a dollar sign (\$) before the variable name to access the variable value. In cases where the variable name might be ambiguous, put braces ({}) around the variable name.

Manipulating Variables

This section describes how to

- List existing variables
- Display variable values
- Assign variable values
- Initialize variables
- Create and change variables
- Use variables
- Remove variables

Listing Existing Variables

Use the `printvar` command to display all of the variables defined in your current session, as well as their values. By default, the `printvar` command displays both application variables and user-defined variables. To display only application variables, use the `report_app_var` command or use the `-application` option with the `printvar` command. To display only user-defined variables, use the `-user_defined` option.

The following example displays all the variables defined in the current session, along with their current value:

```
prompt> printvar
...
compile_advanced_fix_multiple_port_nets = "false"
compile_allow_dw_hierarchical_inverter_opt = "false"
compile_assume_fully_decoded_three_state_busses = "false"
compile_auto_ungroup_area_num_cells = "30"
compile_auto_ungroup_count_leaf_cells = "false"
...
```

The following example displays all the application variables, along with their current value, data type, and default value:

```
prompt> report_app_var
Variable                Value      Type      Default  Constraints
-----
abstraction_ignore_percentage 25      real      25
access_internal_pins      false    string    false
...
```


Displaying Variable Values

To display the values of variables, use the `printvar` command. For application variables, you can also use the `report_app_var` and `get_app_var` commands.

The following example shows the various methods of displaying the value of an application variable:

```
prompt> printvar bus_naming_style
bus_naming_style = "%s[%d]"
prompt> get_app_var bus_naming_style
%s[%d]
prompt> report_app_var bus_naming_style
```

Variable	Value	Type	Default	Constraints
bus_naming_style	%s[%d]	string	%s\[%d\]	

Assigning Variable Values

To assign a value to an application variable, use the `set_app_var` command. The `set_app_var` command ensures that the specified variable is in fact an application variable and also performs data type checking.

For example, to set the search path, use the following command:

```
prompt> set_app_var search_path { ./usr/synopsys/libraries}
./usr/synopsys/libraries
```

To assign a new or initial value to a user-defined variable, use the `set` command. If the variable does not already exist, the tool creates it and sets its initial value to the specified value. If the variable already exists, its value is updated.

For example, to create a user-defined variable named `my_design` and set its value to `TOP_DESIGN`, use the following command:

```
prompt> set my_design TOP_DESIGN
TOP_DESIGN
```

Using Variables

To use a variable's value, you must precede the variable name with a dollar sign (\$). If the variable name might be ambiguous in the command line, put braces ({}) around the variable name. For example,

```
prompt> set clk_period 20
prompt> create_clock -period $clk_period CLK
prompt> set log_dir "./log/"
prompt> report_constraint > "${log_dir}run.log"
```

The command-line editor allows you to press the Tab key to complete variable names automatically on the command line. If the command-line editor cannot find a matching string, it lists all closely matching strings. The command-line editor is enabled by default. To disable this feature, set the `shell.common.enable_line_editing` application option to `false`. For more information, see [Command-Line Editor](#).

Numeric Variable Precision

The precision of a numeric variable depends on how you assign a numeric value to it. A numeric variable becomes a floating number if you use the decimal point; otherwise, it becomes an integer. An integer variable can be treated as a decimal, octal, or hexadecimal number when used in expressions.

To avoid unexpected results, you must be aware of the precision of a numeric variable when using it in an expression. For example, in the following commands, the division operator produces different results when used with integer and floating-point numbers:

```
prompt> set a 10; set b 4.0; set c 4
4
prompt> expr $a/$b ;# floating-point value returned
2.5
prompt> expr $a/$c ;# integer value returned
2
```

The first `expr` command performs floating-point division; the second `expr` command performs integer division. Integer division does not yield the fractional portion of the result. When integer and floating-point variables are used in the same expression, the operation becomes a floating-point operation, and the result is represented as floating point.

To force floating-point operations for integer values, multiply one of the first values to be evaluated in the expression by 1.0 to convert it to floating-point:

```
prompt> expr (1.0*$a)/$c ;# floating-point result returned
2.5
```

Take care that this floating-point conversion occurs before operations are performed on the integer values, or the results might not be as expected:

```
prompt> expr 1.0*($a/$c) ;# ($a/$c) evaluated in integer context
2.0
prompt> expr 0.0+$a/$c ;# ($a/$c) evaluated in integer context
2.0
```

See Also

- [SolvNetPlus article 000014175](#), “Why Does TCL Give Wrong Calculation Results?” for more details on managing floating-point precision in Tcl expressions

Removing Variables

Use the `unset` command to remove a user-defined variable. You cannot remove application variables; if you attempt to remove an application variable, the tool issues an error message.

4

Control Flow

The Tcl control flow statement commands—`if`, `while`, `for`, `foreach`, `break`, `continue`, and `switch`—determine the execution order of other commands. Control flow statements are used primarily in command scripts, such as to check whether a previous command executed successfully. Any command can be used within a control flow statement, including additional nested control flow statements.

Control flow statements are described in the following sections:

- [Condition Expressions](#)
- [Conditional Command Execution](#)
- [Loops](#)

Condition Expressions

All control flow statements evaluate a *conditional expression*. The conditional expression is enclosed in curly braces (`{ }`) and is implicitly evaluated using the `expr` command. The evaluation result is treated as Boolean true or false as described in [Table 14](#).

Table 14 Boolean Equivalents of Non-Boolean Types

Evaluation result	False	True
Integer	0	Any value but 0
Float	0.0	Any value but 0.0
Boolean string	false, no (case-insensitive)	true, yes (case-insensitive)

Values not listed in the table, such as lists or non-Boolean string values (including empty strings) result in an error:

```
prompt> set string_to_build {}
prompt> while {! $string_to_build} {...}
Error: expected boolean value but got ""
      Use error_info for more info. (CMD-013)
```

However, strings can be compared using the `eq` (equal) or `ne` (not equal) Tcl string comparison operators:

```
prompt> set string_to_build {}  
prompt> while {$string_to_build eq {}} {...}
```

Conditional Command Execution

The conditional command execution statements are

- `if`
- `switch`

The `if` and `switch` commands provide a way to select for execution one block of script from several blocks.

if Statement

An `if` command requires two arguments; in addition, it can be extended to contain `elseif` and `else` arguments. The required arguments are

- An expression to evaluate
- A script to conditionally execute based on the result of the expression

The basic syntax of the `if` command is

```
if {expression} {  
    script  
}
```

The `if` command evaluates the expression, and if the result is not zero, the script is executed.

The `if` command can be extended to contain one or more `elseif` arguments and a final `else` argument. An `elseif` argument requires two additional arguments: an expression and a script. An `else` argument requires only a script.

The basic format is as follows:

```
if {expression1} {  
    script1  
} elseif {expression2} {  
    script2  
} else {  
    script3  
}
```

The following example shows how to use the `elseif` and `else` arguments:

```
if {$x == 0} {  
    echo "Equal"  
}  
elseif {$x > 0} {  
    echo "Greater"  
}  
else {  
    echo "Less"  
}
```

The `elseif` and `else` arguments appear on the same line with the closing brace (`}`). This syntax is required because a new line indicates a new command. If the `elseif` argument is on a separate line, it is treated as a command, which it is not.

switch Statement

The `switch` command provides a more compact encoding alternative to using an `if` command with many `elseif` arguments. The `switch` command tests a value against a number of string patterns and executes the script corresponding to the first pattern that matches.

The syntax of the `switch` statement is

```
switch test_value {  
    pattern1 {script1}  
    pattern2 {script2}  
    ...  
}
```

The expression `test_value` is the value to be tested. The `test_value` expression is compared one by one to the patterns. Each pattern is paired with a statement, procedure, or command script. If `test_value` matches a pattern, the script associated with the matching pattern is run.

The `switch` statement supports three forms of pattern matching:

- The `test_value` expression and the pattern match exactly (`-exact`).
- The pattern uses wildcards (`-glob`).
- The pattern is a regular expression (`-regexp`).

Specify the form of pattern matching by adding an argument (`-exact`, `-glob`, or `-regexp`) before the `test_value` option. If no pattern matching form is specified, the pattern matching used is equivalent to `-glob`.

If the last pattern specified is default, it matches any value.

If the script in a pattern and script pair is `(-)`, the script in the next pattern is used.

The following example uses the value of the `vendor_library` variable to determine the maximum delay value.

```
switch -exact $vendor_library {  
    Xlib {set_max_delay 2.8 [all_outputs]}  
    Ylib { - }  
    Zlib {set_max_delay 3.1 [all_outputs]}  
    default {set_max_delay 3.4 [all_outputs]}  
}
```

Loops

The loop statements are

- `while`
- `for`
- `foreach`
- `foreach_in_collection`

The `while`, `for`, and `foreach` commands provide a way to repeat a block of script (looping). The `break` and `continue` commands are used in conjunction with looping to change the normal execution order of loops.

while Statement

The `while` statement repeatedly executes a single set of commands while a given condition is true.

The syntax of the `while` statement is

```
while {expression} {while_command while_command ... }
```

As long as the expression is true, the set of commands specified by the `while_command` arguments are repeatedly executed.

The expression becomes a Boolean variable.

If a `continue` statement is encountered in a while loop, the tool immediately starts over at the top of the while loop and reevaluates the expression.

If a `break` statement is encountered, the tool terminates the loop and resumes execution at the first command after the loop.

For example, the following `while` command prints squared values from 0 to 10:

```
set p 0
while {$p <= 10} {
    echo "$p squared is: [expr $p * $p]"
    incr p
}
```

for Statement

The `for` command has four arguments:

- An initialization expression
- A loop-termination expression
- A reinitialization expression
- The script to execute for each iteration of the `for` loop

The syntax of the `for` statement is

```
for {init} {test} {reinit} {
    body
}
```

The `for` loop runs *init* as a Tcl script, then evaluates *test* as an expression. If *test* evaluates to a nonzero value, *body* is run as a Tcl script, *reinit* is run as a Tcl script, and *test* is reevaluated. As long as the reevaluation of *test* results in a nonzero value, the loop continues. The `for` statement returns an empty string.

The following example prints the squared values from 0 to 10:

```
for {set p 0} {$p <= 10} {incr p} {
    echo "$p squared is: [expr $p * $p]"
}
```

foreach Statement

The `foreach` command iterates over the elements in a list. It has three arguments:

- A variable name
- A list
- A script to execute

The `foreach` statement runs a set of commands one time for each value assigned to the specified variable.

The syntax is

```
foreach variable_name list {  
    foreach_command  
    foreach_command  
    ...  
}
```

The `foreach` statement sets *variable_name* to each value represented by the *list* expression and executes the identified set of commands for each value. The *variable_name* variable retains its value when the `foreach` loop ends.

A carriage return or semicolon must precede the closing brace of the `foreach` statement. The following example shows how you use a `foreach` statement.

```
set x {a b c}  
foreach member $x {  
    printvar member  
}  
member = "a"  
member = "b"  
member = "c"
```

Use the `foreach_in_collection` statement to traverse design objects, rather than the `foreach` statement.

foreach_in_collection Statement

The `foreach_in_collection` statement is a specialized version of the `foreach` statement that iterates over the elements in a specified collection. The syntax is

```
foreach_in_collection collection_item collection {  
    body  
}
```

where *collection_item* is set to the current member of the collection as the `foreach_in_collection` command iterates over the members, *collection* is a collection, and *body* is a Tcl script executed for each element in the collection.

For example, to print the load attribute for all ports in the design, enter

```
foreach_in_collection eachport [get_ports *] {  
    set loadval [get_attribute [get_object_name $eachport] load]  
    printvar loadval  
}
```

Loop Control and Termination

The loop termination statements are `continue` and `break`.

The difference between `continue` and `break` statements is that the `continue` statement causes command execution to continue by evaluating the next loop iteration, whereas the `break` statement causes command execution to break out of the loop entirely.

In the case of nested loops, only the innermost loop is affected.

continue Statement

Use the `continue` statement to skip the remainder of the current loop iteration and begin evaluating the next iteration.

In the following example, the `continue` statement causes the printing of only the squares of even numbers between 0 to 10:

```
set p 0
while {$p <= 10} {
    if {$p % 2} {
        incr p
        continue
    }
    echo "$p squared is: [expr $p * $p]"
    incr p
}
```

break Statement

Use the `break` statement to immediately terminate the loop and resume execution with the first statement after the loop.

In the following example, a list of file names is scanned until the first file name that is a directory is encountered. The `break` statement is used to terminate the `foreach` loop when the first directory name is encountered.

```
foreach f [which {VDD.ave GND.tech p4mvn2mb.idm}] {
    echo -n "File $f is "
    if { [file isdirectory $f] == 0 } {
        echo "NOT a directory"
    } else {
        echo "a directory - breaking out of loop"
        break
    }
}
```

5

Working With Procedures

A procedure is a named block of commands that performs a particular task or function. With procedures, you create new commands by using existing Tcl and Synopsys commands. This chapter shows you how to create procedures, and it describes how to use Synopsys procedure extensions.

This chapter contains the following sections:

- [Creating Procedures](#)
- [Extending Procedures](#)
- [Displaying the Procedure Body and Arguments](#)

Creating Procedures

You use the `proc` command to create a procedure. The syntax of the `proc` command is

```
proc name args body
```

The *name* argument names your procedure. You cannot use the name of an existing Tcl or Synopsys command. You can, however, use the name of an existing procedure, and if a procedure with the name you specify exists, your procedure replaces the existing procedure.

The arguments to a procedure are specified in the *args* argument, and the script that makes up a procedure is contained in the *body* argument. You can create procedures without arguments also. Arguments to a procedure must be scalar variables; consequently you cannot use arrays as arguments to a procedure. (For a technique to overcome this limitation, see [Using Arrays With Procedures](#).)

The following is a procedure example:

```
# procedure max
# returns the greater of two values
proc max {a b} {
    if {$a > $b} {
        return $a
    }
    return $b
}
```

You invoke this procedure as follows:

```
prompt> max 10 5
```

To save the result of the procedure, set a variable to its result. For example,

```
prompt> set bigger [max 10 5]
```

When a procedure terminates, the return value is the value specified in a `return` command. If a procedure does not execute an explicit `return` command, the return value is the value of the last command executed in the body of the procedure. If an error occurs while the body of the procedure is being executed, the procedure returns that error.

The `return` command causes the procedure to return immediately; commands that come after the `return` command are not executed.

Variable Scope

Variable scope determines the accessibility of a variable when it is used in scripts and procedures. In Tcl, the scope of a variable can be either local or global. When working with scripts and procedures, you must be aware of a variable's scope to ensure that it is used properly.

When a procedure is invoked, a local variable is created for each argument of the procedure. Local variables are accessible only within the procedure from which they are created, and they are deleted when the procedure terminates. A variable created within the procedure body is also a local variable.

Variables created outside of procedures are called global variables. You can access a global variable from within a procedure by using the `global` command. The `global` command establishes a connection to the named global variable, and references are directed to that global variable until the procedure terminates. (For more information, see the `global` man page.)

You can also access variables that are outside the scope of a procedure by using the `upvar` command. This command is useful for linking nonscalar variables (for example, arrays) to a procedure because they cannot be used as arguments to a procedure. For more information, see the man page for the `upvar` command.

It is possible to create a local variable with the same name as a global variable and to create local variables with the same name in different procedures. In each case, these are different variables, so changes to one do not affect the other.

For example,

```
# Variable scope example
set ga 5
set gb clock_ext
```

```
proc scope_ex1 {a b} {  
    echo $a $b  
    set gb 100  
    echo $gb  
}  
  
proc scope_ex2 {a b} {  
    echo $a $b  
    set gb 4.25  
    echo $gb  
}
```

In this script example, `ga` and `gb` are global variables because they are created outside of the `scope_ex1` and `scope_ex2` procedures. The variable name `gb` is also used within the `scope_ex1` and `scope_ex2` procedures. Within these procedures, `gb` is a local variable. The three instances of `gb` exist as three different variables. A change to one instance of `gb` does not affect the others.

Argument Defaults

You can specify the default for one or more of the arguments of a procedure. To set up a default for an argument, you place the arguments of the procedure in a sublist that contains two elements: the name of the argument and its default. For example,

```
# procedure max  
# returns the greater of two values  
proc max {{a 0} {b 0}} {  
    if {$a > $b} {  
        return $a  
    }  
    return $b  
}
```

In this example, you can invoke `max` with two or fewer arguments. If an argument is missing, its value is set to the specified default, 0 in this case.

With this procedure, the following invocations are all valid:

```
max  
max arg1  
max arg1 arg2
```

You do not have to surround nondefault arguments within braces. For example,

```
# procedure max  
# returns the greater of two values  
proc max {a {b 0}} {  
    ...  
}
```

You should also consider the following points when using default arguments:

- If you do not specify a particular argument with a default, you must supply that argument when the procedure is invoked.
- If you use default arguments, you must place them after all nondefault arguments.
- If you specify a default for a particular argument, you must specify a default for all arguments that follow.
- If you omit an argument, you must omit all arguments that follow.

Variable Numbers of Arguments

You can create procedures with variable numbers of arguments if you use the special argument `args`. This argument must be positioned as the last argument in the argument list; arguments preceding `args` are handled as described in the previous sections.

Additional arguments are placed into `args` as a list. The following example shows how to use a varying number of arguments:

```
# print the square of at least one number
proc squares {num args} {
    set nlist $num
    append nlist " "
    append nlist $args
    foreach n $nlist {
        echo "Square of $n is [expr $n*$n]"
    }
}
```

Using Arrays With Procedures

When using an array with a procedure, you can make the array a global variable, or you can use the `get` and `set` options of the `array` command to manipulate the array so that it can be used as an argument or as the return value of a procedure. [Example 2](#) demonstrates the latter technique.

Example 2 Passing an Array to a Procedure

```
proc my_proc { bar_list } {
    # bar was an array in the main code
    array set bar_array $bar_list;
    # manipulate bar_array
    return [array get bar_array];
}
set orig_data(one) {two};
set orig_data(alpha) {green};
array set new_data [my_proc [array get orig_data]];
```

General Considerations for Using Procedures

Keep in mind the following points when using procedures:

- Procedures can use Tcl and Synopsys commands.
- Procedures can use other procedures provided that they contain supported Tcl and Synopsys commands.
- Procedures can be recursive.
- Procedures can contain local variables and can reference variables outside their scope (see [Variable Scope](#)).

Extending Procedures

This section describes the `define_proc_attributes` and `parse_proc_arguments` commands. These commands add extended functionality to the procedures you create. With these commands, you can create procedures with the same help and semantic attributes as Synopsys commands.

When you create a procedure, it has the following intrinsic attributes:

- The body of the procedure can be viewed with the `info body` command.
- The procedure can be modified.
- The procedure name can be abbreviated according to the value of the `sh_command_abbrev_mode` variable.
- The procedure is placed in the Procedures command group.

Note:

The procedure does not have help text.

By using the `define_proc_attributes` command, you can

- Specify help text for the command
- Specify rules for argument validation
- Prevent procedure view and modification
- Prevent procedure name abbreviation
- Specify the command group in which to place the procedure

You use the `parse_proc_arguments` command in conjunction with the `define_proc_attributes` command to enable the `-help` option for a procedure and to support procedure argument validation.

Using the `define_proc_attributes` Command

You use the `define_proc_attributes` command to define and change the attributes of a procedure.

The syntax is

```
define_proc_attributes proc_name
    [-info info_text]
    [-define_args arg_defs]
    [-command_group group_name]
    [-hide_body]
    [-hidden]
    [-permanent]
    [-dont_abbrev]
```

The arguments are defined as follows:

`proc_name`

Specifies the name of the procedure to extend.

`-info info_text`

Specifies the quick-help text that is used in conjunction with the `help` command and the procedure's `-help` option. The text is limited to one line.

`-define_args arg_defs`

Specifies the help text for the procedure's arguments and defines the procedure arguments and their attributes.

For information about using the `-define_args` argument within a procedure, see [Using the `parse\_proc\_arguments` Command](#).

`-command_group group_name`

Specifies the command group of the procedure. The default command group is `Procedures`. This attribute is used in conjunction with the `help` command.

For more information, see [Getting Help on Commands](#).

`-hide_body`

Hides the body of the procedure. The procedure body cannot be viewed by using the `body` option of the `info` command. This attribute does not affect the `info` command when the `args` option is used.

`-hidden`

Hides the procedure so the `help` command cannot access the help page, and the `info proc` command cannot access the body of the procedure.

`-permanent`

Prevents modifications to the procedure.

`-dont_abbrev`

Prevents name abbreviation for the procedure, regardless of the value of the `sh_command_abbrev_mode` variable.

You use the `-define_args` option to specify quick-help text for the procedure's arguments and to define the data type and attributes of the procedure's arguments. The `-define_args` argument is a list of lists. For more information, see [Lists](#). Each list element specifies the attributes for a procedure argument.

Each list element has the following format:

arg_name option_help value_help data_type attributes

arg_name

Specifies the name of the procedure argument.

option_help

Specifies a short description of the argument for use with the procedure's `-help` option.

value_help

For positional arguments, specifies the argument name; otherwise, is a one-word description for the value of a dash option. This parameter has no meaning for a Boolean option.

data_type

Specifies the data type of the argument; the *data_type* parameter can be one of the following: `string` (the default), `list`, `boolean`, `int`, `float`, or `one_of_string`. This parameter is optional.

attributes

Specifies additional attributes for an argument. This parameter is optional. The additional attributes are described in [Table 15](#).

Table 15 Additional Argument Attributes

Attribute	Description
required	Specifies a required argument. You cannot use this attribute with the optional attribute.
optional	Specifies an optional argument. You cannot use this attribute with the required attribute.
value_help	Specifies that valid values for one_of_string arguments be shown when the argument help is shown for a procedure. For data types other than one_of_string, this attribute is ignored.
values	Specifies the list of valid values for one_of_string arguments. This attribute is required if the argument type is one_of_string. If you use this attribute with other data types, an error is displayed.

define_proc_attributes Command Example

The following procedure adds two numbers and returns the sum:

```
prompt> proc plus {a b} {return [expr $a + $b]}
prompt> define_proc_attributes plus \
  -info "Add two numbers" \
  -define_args {
    {a "first addend" a string required} \
    {b "second addend" b string required} \
    {"-verbose" "issue a message" "" boolean optional} }
prompt> help -verbose plus
Usage: plus      # Add two numbers
      [-verbose]      (issue a message)
      a              (first addend)
      b              (second addend)
prompt> plus 5 6
11
```

Using the parse_proc_arguments Command

The `parse_proc_arguments` command parses the arguments passed to a procedure that is defined with the `define_proc_attributes` command.

You use the `parse_proc_arguments` command within procedures to support argument validation and to enable the `-help` option. Typically, `parse_proc_arguments` is the first command called within a procedure. You cannot use the `parse_proc_arguments` command outside a procedure.

The syntax is

```
parse_proc_arguments -args arg_list result_array
```

```
-args arg_list
```

Specifies the list of arguments passed to the procedure.

```
result_array
```

Specifies the name of the array in which to store the parsed arguments.

When a procedure that uses the `parse_proc_arguments` command is invoked with the `-help` option, `parse_proc_arguments` prints help information (in the same style as the `-verbose` option of the `help` command) and then causes the calling procedure to return. If any type of error exists with the arguments (missing required arguments, invalid value, and so forth), `parse_proc_arguments` returns an error, and the procedure terminates.

If you do not specify the `-help` option and the specified arguments are valid, the `result_array` array contains each of the argument values subscripted with the argument name. The argument names are not the names of the arguments in the procedure definition; the argument names are the names of the arguments as defined with the `define_proc_attributes` command.

Example

In [Example 3](#), the `argHandler` procedure shows how the `parse_proc_arguments` command is used. The `argHandler` procedure accepts an optional argument of each type supported by `define_proc_attributes`, then prints the options and values received.

Example 3 *argHandler Procedure*

```
proc argHandler {args} {
    parse_proc_arguments -args $args results
    foreach argname [array names results] {
        echo " $argname = $results($argname)"
    }
}

define_proc_attributes argHandler -info "argument processor" \
    -define_args {
        {-Oos "oos help" AnOos one_of_string {required value_help
            {values {a b}}}}
        {-Int "int help" AnInt int optional}
        {-Float "float help" AFloat float optional}
        {-Bool "bool help" "" boolean optional}
        {-String "string help" AString string optional}
        {-List "list help" AList list optional} }
```

Invoking the `argHandler` procedure with the `-help` option generates the following output:

```
prompt> argHandler -help
Usage: argHandler      # argument processor
      -Oos AnOos      (oos help:
                        Values: a, b)
      [-Int AnInt]    (int help)
      [-Float AFloat] (float help)
      [-Bool]         (bool help)
      [-String AString] (string help)
      [-List AList]   (list help)
```

Invoking the `argHandler` procedure with an invalid option generates the following output and causes an error:

```
prompt> argHandler -Int z
Error: value 'z' for option '-Int' not of type 'integer' (CMD-009)
Error: Required argument '-Oos' was not found (CMD-007)
```

Invoking the `argHandler` procedure with valid arguments generates the following output:

```
prompt> argHandler -Int 6 -Oos a
      -Oos = a
      -Int = 6
```

Considerations for Extending Procedures

When using the extended procedure features, keep in mind the following points:

- The `define_proc_attributes` command does not validate the arguments you define by using its `-define_args` option.
- Whenever possible, use the Tcl variable numbers of arguments feature to facilitate the passing of arguments to the `parse_proc_arguments` command. For more information, see [Variable Numbers of Arguments](#).
- If you do not use the `parse_proc_arguments` command, procedures cannot respond to the `-help` option. However, you can always use the `help` command. For example,

```
help procedure_name -verbose
```

Displaying the Procedure Body and Arguments

This section describes the commands you can use to display the body and the arguments of a procedure: the `info` command and the `proc_body` and `proc_args` commands.

You use the `body` option of the `info` command to display the body of a procedure and the `args` option to display the arguments.

You can use the `proc_body` command as an alternative to `info body` and `proc_args` as an alternative to `info args`.

If you use the `-hide_body` option when you define a procedure, you cannot use the `info body` or `proc_body` commands to view the contents of the procedure.

These commands have the following syntax:

```
info body procedure_name
info args procedure_name

proc_body procedure_name
proc_args procedure_name
```

6

Collections of Design Objects

Synopsys Tcl-based tools provide commands that search for and manipulate information in your design. These commands work with collections, which allow design objects to be efficiently identified and specified.

This chapter includes the following sections:

- [Why Use Collections?](#)
- [Working With Collections](#)
- [Working With Attributes](#)
- [Using Name-Based Object Specifications](#)
- [Matching Names of Design Objects During Queries](#)
- [Reporting Collections](#)
- [Writing Collection Information to a File](#)

Why Use Collections?

Many tool commands accept and/or return a set of one or more design objects as part of their operation. Sometimes the reference is a single design object, such as when you create a clock on a port. Other times the reference is to many design objects, such as when you obtain the set of all registers in the design.

The Tcl scripting language provides the *list* construct, which is simply a list of one or more string values in a particular standard Tcl format. Lists can be used to reference design objects, but there are disadvantages:

- Lists are memory-inefficient.

Every name is stored as a complete string. Hierarchy paths are stored repeatedly and redundantly for multiple hierarchical objects.

- Lists are runtime-inefficient.

Each list item must be searched for in the design database using string comparisons. Hierarchical object names require additional processing to identify each hierarchical block, then the leaf object, during the search.

- Lists are type-ambiguous.

List items don't inherently specify an object type. Does {CLK} refer to a port, or a net of the same name?

- Lists cannot provide additional object information.

List items don't provide any information about a design object except its name. You cannot obtain the clock periods of a list of clocks or the area values of a list of cells.

To overcome these drawbacks, Tcl-based Synopsys tools provide *collections* as a method for referencing design objects. A collection is a binary data structure that references a set of one or more design objects. Each design object reference points directly to an internal data structure, which makes collections very fast and memory-efficient, even when working with many thousands of objects.

Every collection object has an `object_class` attribute that tells you what type of object it is. Collections can contain one type of object (homogeneous) or a variety of object types (heterogeneous).

Each object class provides additional attributes that you can use to find more information about design objects of that class. For example, clock objects have a `period` attribute, cell objects have an `area` attribute, and so on. These values are either retrieved directly from the internal data structures or derived on-the-fly by the tool, so they are efficient to query.

Working With Collections

A collection is a set of design objects such as cells, nets, or libraries. You create, view, and manipulate collections by using commands provided specifically for working with collections. The following sections describe how to work with collections:

- [Creating Collections](#)
- [Saving Collections](#)

- [Displaying Objects in a Collection](#)
- [Adding Objects to a Collection](#)
- [Removing Objects From a Collection](#)
- [Comparing Collections](#)
- [Extracting Individual Objects From a Collection](#)
- [Iterating Over a Collection](#)

Creating Collections

Typically, you create collections with the `get_*` and `all_*` commands. For example, to create a collection that contains the cells that begin with `o` and reference an FD2 library cell, use the following command:

```
prompt> get_cells {o*} -filter {ref_name == FD2}
{o_reg1 o_reg2 o_reg3 o_reg4}
```

Although the returned result looks like a list, it is not. A collection is referenced by a *collection handle*, which is simply a string value that the tool associates with the collection's internal data structure. When a collection is the returned result at an interactive prompt, the tool shows the collection contents instead of the collection handle for convenience. Collections returned by commands during script execution are not printed.

Most command arguments that accept design objects support collections.

Collections can be temporary or persistent throughout a session. If you use a nested collection command as an argument, the collection created by the nested collection command persists only within the scope of the command. For example, to set the `size_only` attribute on cells `i1` and `i2`, use the following command:

```
prompt> set_size_only [get_cells {i1 i2}] true
1
```

If you want the collection to persist in the current session, assign it to a variable. You can then use the collection variable as the argument value. For example,

```
prompt> set my_pins [get_pins o*/CP]
{o_reg1/CP o_reg2/CP}
prompt> get_cells -of_objects $my_pins
{o_reg1 o_reg2}
```

The collection persists as long as the variable references its handle. If you unset the variable, the collection is freed. For more information, see [Saving Collections](#).

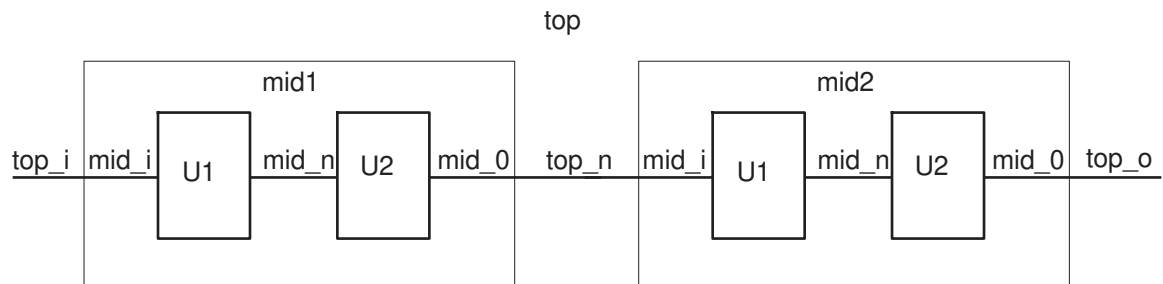
Collection commands often have options that provide fine control over object acquisition. For example, the `get_pins` command provides a `-leaf` option that controls whether

the tool considers only the current net segment or the entire net when getting pins of a hierarchical net:

```
prompt> get_pins -of_objects [get_nets NET1] ;# only connected to NET1
{i2/a reg1/QN}
```

```
prompt> get_pins -leaf -of_objects [get_nets NET1] ;# entire hier network
{i2/U1/A i2/U2/A reg1/QN}
```

The `get_nets` command also contains options that control how net segments are returned for hierarchical nets.



```
prompt> get_nets -top_net_of_hierarchical_group
{top_itop_otop_n}
```

```
prompt> get_nets -top_net_of_hierarchical_group -segments
{top_itop_otop_n}
```

```
prompt> get_nets -segments
{top_itop_otop_nmid1/mid_imid2/mid_omid1/mid_o mid2/mid_i}
```

```
prompt> get_nets -hierarchical
{top_ntop_itop_omid2/mid_nmid2/mid_imid2/mid_o
mid1/mid_nmid1/mid_imid1/mid_o}
```

```
prompt> get_nets -top_net_of_hierarchical_group -segments top_n
{top_n}
```

```
prompt> get_nets -segments top_n
{top_nmid2/mid_imid1/mid_o}
```

```
prompt> get_nets -hierarchical top_n
{top_n}
```

```
prompt> get_nets -top_net_of_hierarchical_group top_n
{top_n}
```

Saving Collections

To save a collection, store the result of the command that creates the collection in a variable. For example, to create a collection variable named `RX_ports` that contains all bits of a bused port `RX`, use the following command:

```
prompt> set RX_ports [get_ports RX[*]]
```

You can pass the variable directly to a command that operates on the collection, as shown in the following example:

```
prompt> set_input_delay -clock myclock 2.0 $RX_ports  
1
```

If you save a collection in a variable and then later remove the variable definition by using the `unset` command, you also remove the collection. For example, to remove the `RX_ports` variable and the associated collection, use the following command:

```
prompt> unset RX_ports
```

Accessing Collections

For tools that use a “current design” concept, changing the current design with the `current_design` command might invalidate existing collections. In this case, you can access the collections only within the design context in which they were created.

Some tools, such as Design Compiler, automatically re-create the deleted collections when you change the current design back to the design in which you originally defined the collections. To reduce the runtime for the `current_design` command, use the `unset` command to delete the collections that you no longer need.

For example,

```
dc_shell> set my_pins [get_pins ...]  
# do things with $my_pins  
dc_shell> unset my_pins
```

The Design Compiler tool does not re-create collections that contain the following design objects:

- Clusters
- Scan paths
- Timing paths

Other tools, such as the IC Compiler II tool, do not delete collections when changing design focus.

Displaying Objects in a Collection

All commands that return collections implicitly display the collection contents when the collection is returned at the command prompt. However, for more flexibility, you can use the `query_objects` command to display objects in a collection.

The `query_objects` command generates output that is similar to the interactive output of a collection command. The query results are formatted as a Tcl list (for example, {a b c d ...}), so that you can directly examine the results.

For example, to display all of the ports that start with the string "in", use the following command:

```
prompt> query_objects [get_ports in*]
{in0 in1 in2}
```

The `query_objects` command also allows you to search the design database directly. For example, the following command returns the same information as the previous `query_objects` command:

```
prompt> query_objects -class port in*
{in0 in1 in2}
```

If the display is truncated, you see an ellipsis (...) as the last element:

```
prompt> query_objects $all_cells
{U129 U130 U131 U132 U133 U134 U135 U136 U137 U138 U139 U140 U141 U142
U143 U144 U145 U146 U147 U148 U149 U150 U151 U152 U153 U154 U155 U156
U157 U158 U159 U160 U161 U162 U163 U164 U165 U166 U167 U168 ...}
```

To control the number of elements displayed, use the `-truncate` option.

You can change the default truncation limit (for both interactive collection reporting and the `query_objects` command) by setting the `collection_result_display_limit` variable to a different value; the default is 100. For more information, see the man page.

To report the number of objects in a collection (instead of the object names), use the `sizeof_collection` command:

```
prompt> sizeof_collection $all_cells
23856
```

For more detailed collection reports, including customizable reports, you can use the `report_collection` command. See [Reporting Collections](#).

Adding Objects to a Collection

There are two ways to add objects to a collection, each suited to a particular purpose:

- Adding objects to an existing collection

You can use the `append_to_collection` command to add objects to an existing collection. Provide the name of the collection variable, then the objects to be added:

```
prompt> append_to_collection all_my_ports $this_port
```

Because the new objects are added directly to the existing named collection at the data structure level, this operation is very fast. Reference the existing collection by its name (without "\$" to reference its value).

- Creating a new collection from existing collections

You can use the `add_to_collection` command to combine two collections together and return the resulting collection as a new collection. For example,

```
prompt> set possible_sources \  
      [add_to_collection $clock_sources $reset_sources]
```

Note that both collection references use "\$" to reference the collection values, not their names. The object references in the input collections are duplicated to create the returned collection.

Note:

Use `add_to_collection` only when combining two collections to form a third. To add items to an existing collection, use the `append_to_collection` command instead.

You can also create list variables that contain references to several collections. For example,

```
prompt> set a [get_ports P*]  
{PORT0 PORT1}  
prompt> set b [get_cells reg*]  
{reg0 reg1}  
prompt> set c "$a $b"  
_sel27 _sel28
```

However, this method of combining collections is not recommended.

Removing Objects From a Collection

You use the `remove_from_collection` command to remove objects from a collection. The `remove_from_collection` command creates a new collection that includes the

objects in the original collection minus the specified objects. If the operation results in zero elements, the command returns an empty string. The original collection is not modified.

For example, you can use the following command to create a collection containing all input ports except CLOCK:

```
prompt> set data_ports [remove_from_collection [all_inputs] {CLOCK}]
{in1 in2}
```

You can specify a list of objects or collections to remove. The object class of each element to be removed must be the same as the original collection objects. For example, you cannot remove a port collection from a cell collection.

You can also remove objects from a collection by using a filter expression that limits the objects in the collection. For more information, see [Filtering Collections by Attribute Values](#).

Comparing Collections

You use the `compare_collections` command to compare the contents of two collections. If the two collections contain the same set of objects, with no unmatched objects in either one, the `compare_collections` command returns zero; otherwise it returns a nonzero value.

For example,

```
prompt> compare_collections [get_cells *] [get_cells *]
0
```

By default, the order of the objects in each collection does not matter. You can make the comparison order-dependent by using the `-order_dependent` option.

To determine if a collection is empty, compare it against an empty string using Tcl string comparison:

```
prompt> if {$my_cells eq {}} { # ...it is empty... }

prompt> if {$my_cells ne {}} { # ...it is not empty... }
```

To determine if all objects in one collection (the "needle") exist in another (the "haystack"), use the `remove_from_collection` command as follows:

```
prompt> if {[remove_from_collection $needle $haystack] eq {}} {
    # all objects in $needle exist in $haystack
}
```

To determine what objects are common between two collections, if any, use the `remove_from_collection -intersect` command.

```
prompt> remove_from_collection -intersect $fanin_pins $fanout_pins
```

If there are no common objects, the command returns an empty collection.

Extracting Individual Objects From a Collection

Collections are ordered. The objects in a collection are numbered 0 through $n-1$, where n is the size of the collection.

You can use the `index_collection` command to return individual objects by index number:

- To extract single objects, specify a single index value:

```
prompt> index_collection $my_ports 3
{A[12]}
```

(This corresponds to the Tcl `lindex` command for lists.)

- To extract a range of objects, specify the lower and upper index values of a range:

```
prompt> index_collection $my_ports 3 5
{A[12] A[11] A[10]}
```

(This corresponds to the Tcl `lrange` command for lists.)

Just as with the corresponding Tcl commands, you can specify `end` or `end-value` for the index values:

```
prompt> index_collection $my_ports end
{test_se}
prompt> index_collection $my_ports end-2 end
{test_si6 test_si7 test_se}
```

Note:

Although commands such as `get_cells` might not *document* how their collections are ordered, they do create them in a deterministic order. In other words, the same command executed multiple times (such as `get_cells *`) creates the same collection each time.

Iterating Over a Collection

You use the `foreach_in_collection` command to iterate over each element in a collection. The `foreach_in_collection` command can be nested within other control structures, including another `foreach_in_collection` command.

During each iteration, the iteration variable is set to a collection of exactly one object. For example,

```
prompt> foreach_in_collection this_cell [get_cells *] { ... }
```

You cannot use the `foreach` command to iterate over a collection.

Iteratively Adding Objects to a Collection

To iteratively add objects to a collection, use the `append_to_collection` command to directly add objects in-place to that collection. For example,

```
# start with an empty collection for object accumulation
set qualifying_cells {}

# loop through a collection and conditionally add
# qualifying objects
foreach_in_collection this_cell $all_cells {
    if {...some criteria...} {
        append_to_collection qualifying_cells $this_cell
    }
}
```

Never use the `add_to_collection` command to iteratively add objects to a collection, as there is a significant performance penalty for continually creating and destroying temporary collections within a loop.

Iteratively Removing Objects From a Collection

To iteratively remove objects from a collection, use the `append_to_collection` command to create the set of objects to remove from that collection, then remove them after the loop. For example,

```
# start with an empty collection for object accumulation
set cells_to_remove {}

# loop through a collection and conditionally
# accumulate qualifying objects to remove
foreach_in_collection this_cell $all_cells {
    if {...some criteria...} {
        append_to_collection cells_to_remove $this_cell
    }
}

# now remove the accumulated objects from the full collection
set all_cells [remove_from_collection $all_cells $cells_to_remove]
```

Never use the `remove_from_collection` command to iteratively remove objects from a collection, as there is a significant performance penalty for continually creating and destroying temporary collections within a loop.

Working With Attributes

Collection objects have attributes that provide additional information. Collection attributes are described in the following sections:

- [Setting and Querying Attribute Values](#)
- [Filtering Collections by Attribute Values](#)
- [Chained Attribute Access](#)
- [User-Defined Attributes](#)
- [Derived User Attributes](#)

Setting and Querying Attribute Values

Collection objects have *attributes* that you can query by name for additional information. Each object type (cells, pins, and so on) provides its own set of attributes. Each attribute returns data of a particular type: Boolean, integer, float, string, or collection.

For example, every collection object has an `object_class` attribute that returns a string indicating what type of object it is:

```
prompt> get_attribute [get_cells {U123}] object_class
cell
```

Most named design objects have a `full_name` attribute that returns the name string:

```
prompt> get_attribute [get_cells {U123}] full_name
U123
```

For collections of multiple objects, attribute values are returned in the same order as the collection objects:

```
prompt> get_attribute [get_cells {U123 U124 Z_reg}] is_sequential
false false true
```

Attributes can be defined or undefined on objects (depending on the tool and the attribute):

```
prompt> get_attribute [get_pins U123/Z] function
!(A B)
prompt> get_attribute [get_pins U123/A] function
Warning: Attribute 'function' does not exist on pin 'U123/A'. (UID-101)
```

Some attributes return their data as a collection:

```
prompt> set path [get_timing_paths ...]
_sel28
prompt> sizeof_collection [get_attribute $p startpoint]
```



```
1
prompt> sizeof_collection [get_attribute $p points]
7
```

Some tools have additional attribute-related commands, such as for setting attributes and defining and setting user attributes. To see the list of commands in your tool, type:

```
prompt> help *attribute*
```

Filtering Collections by Attribute Values

You can select specific objects from a collection by *attribute filtering*. You can perform comparisons on attribute values, check for the existence or absence of attributes, and even create complex filtering expressions using Boolean operators.

There are two ways to filter collection objects by attribute:

- `filter_collection collection filter_expression`

Use this method to create a new filtered collection from an existing collection.

- `get_* ... -filter filter_expression`

Many `get_*` commands have a `-filter` option that filter on-the-fly while obtaining the design objects. This method is much more efficient than obtaining all unfiltered design objects, and then filtering that full collection with `filter_collection`.

Attribute filtering allows you to filter collection objects more quickly and efficiently than using the `foreach_in_collection` command for object-by-object comparison.

Using Filter Expressions

A filter expression is a set of logical expressions describing the constraints you want to place on the collection objects. A filter expression compares the value of a named attribute (such as `area` or `pin_direction`) with a value (such as 43 or `input`) by means of a relational operator in the following format:

```
left_side relational_operator right_side
```

For example, the following filter expression selects all hierarchical objects whose area attribute is less than 12 units:

```
{is_hierarchical == true && area < 12}
```

[Table 16](#) shows the relational operators that you can use in filter expressions.

Table 16 Relational Operators

Syntax	Operator description	Supported types
<code>a<b</code>	1 if a is less than b, 0 otherwise	float, integer, string
<code>a>b</code>	1 if a is greater than b, 0 otherwise	float, integer, string
<code>a<=b</code>	1 if a is less than or equal to b, 0 otherwise	float, integer, string
<code>a>=b</code>	1 if a is greater than or equal to b, 0 otherwise	float, integer, string
<code>a==b</code>	1 if a is equal to b, 0 otherwise	float, integer, string, Boolean
<code>a!=b</code>	1 if a is not equal to b, 0 otherwise	float, integer, string, Boolean
<code>a=~b</code>	1 if a pattern-matches b, 0 otherwise ²	string
<code>a!~b</code>	1 if a does not pattern-match b, 0 otherwise ²	string

Filter expressions also provide functions that return information about attributes. These functions are supported in *left_side* arguments.

Table 17 shows the available attribute functions.

Table 17 Attribute Functions

Function syntax	Result description	Return type
<code>defined(attribute)</code>	true if <i>attribute</i> is defined, false otherwise	Boolean
<code>undefined(attribute)</code>	true if <i>attribute</i> is not defined, false otherwise	Boolean
<code>sizeof(attribute)</code>	The number of objects in the collection returned by <i>attribute</i> (requires that <i>attribute</i> returns a collection)	integer

For an attribute that returns a collection, you can evaluate the attribute using the functions in Table 17, but you cannot compare its contents using the relational operators in Table 16. For single-item collections, use nested attributes (see [Setting and Querying Attribute Values](#)) to access the item name. For multiple-item collections, look for a name-based equivalent to the attribute, then compare using pattern matching.

You can combine relational expressions by using logical AND (AND or &&) or logical OR (OR or ||) operators. You can group logical expressions with parentheses to enforce order; otherwise the order is left to right.

2. Pattern is anchored at both ends of "a". Glob-style pattern matching is used by default; use the `-regex` option of the command to use regular-expression pattern matching instead.

When using a filter expression as an argument to a command, you must enclose the entire filter expression in quotation marks or braces:

```
filter_collection $cells \  
  {is_hierarchical == true && dont_touch == true} ;# static expression  
  
filter_collection $cells \  
  "is_hierarchical == true && ref_name == $my_design" ;# var substitution
```

However, if you use a string variable to pass the filter expression, you do not need to enclose the string in quotation marks:

```
set my_filter "is_hierarchical == true && ref_name == $my_design"  
filter_collection $cells $my_filter
```

Collection filtering accepts an attribute preceded by the at (@) symbol in the *right_side* expression, allowing you to filter a collection of objects based on attribute comparison within each object.

For example,

```
prompt> filter_collection $my_pins \  
        "actual_rise_transition_max > @constraining_max_transition"
```

or

```
prompt> get_pins -hierarchical * -filter \  
        "actual_rise_transition_max > @constraining_max_transition"
```

Note:

Preceding the attribute name with the at (@) symbol is

- Mandatory in the *right_side* expression
- Optional in the *left_side* expression

When filtering by string value, if the *right_side* string value begins with an at (@) symbol, use quotation marks around the string value so it is not treated as an attribute name.

A filter expression can fail to parse because of the following reasons:

- Invalid syntax
- Invalid attribute name
- A type mismatch between an attribute and its comparison value

To determine the valid attributes for an object type, use the `list_attributes -application -class object_type` command. This command generates a list of all application attributes that apply to the specified object type. For the list of supported object

types, see the man page. one of the following object types: design, port, cell, clock, pin, net, or lib.

Chained Attribute Access

For attributes that return collection objects, you can use *chained attribute access* to perform subsequent attribute queries of those objects. This allows you to “chain together” a series of attributes for short yet powerful queries.

To do this, specify multiple attribute names separated by a period (“.”) character:

```
prompt> get_attribute $path points.object.full_name
DATA_IN U1/A U1/Z U2/A U2/Z FF1/D
```

All attributes specified in the chain must return collection objects, except for the last attribute (which can return any type of data).

If any attribute query in the chain returns multiple objects, then the next query in the chain operates on each object in the set, with the results concatenated in that order:

```
prompt> get_attribute $clock_path points.object.full_name
CLK3 UCLKMUX/C UCLKMUX/Z FF1/CP
prompt> get_attribute $clock_path points.object.clocks
CLK3 CLK3 CLK1 CLK2 CLK3 CLK1 CLK2 CLK3
prompt> get_attribute $clock_path points.object.clocks.period
30 30 10 20 30 10 20 30
```

The `sort_collection` and `filter_collection` commands also support chained attribute access:

```
prompt> set matching_point \
    [filter_collection $timing_path_points \
      "object.full_name == $saved_name"]
prompt> set path_points_sorted_by_capacitance \
    [sort_collection $timing_path_points \
      object.net.total_capacitance_max]
```

Note:

In the Design Compiler tool, the `sort_collection` and `filter_collection` commands support chained attribute access, but the `get_attribute` command does not.

User-Defined Attributes

User-defined attributes allow you to define your own attributes that store values on collection objects.

The `define_user_attribute` command allows you to define a user-defined attribute for particular object classes and of a particular data type:

```
prompt> define_user_attribute \  
        frequency \  
        -classes {clock} \  
        -type double
```

Depending on the tool, you then use the `set_attribute` (most tools) or `set_user_attribute` (PrimeTime) command to set attribute values on collection objects:

```
prompt> set_attribute [get_clocks slow_clk] frequency 62.5  
prompt> set_attribute [get_clocks system_clk] frequency 100  
prompt> set_attribute [get_clocks rx_clk] frequency 125  
prompt> set_attribute [get_clocks tx_clk] frequency 125
```

User-defined attributes can be queried and filtered with, just like regular application attributes:

```
prompt> get_attribute [all_clocks] frequency  
62.5 100 125 125  
prompt> get_clocks * -filter {frequency > 100}  
rx_clk tx_clk
```

Note:

Not all tools support user-defined attributes. To check if your tool does, use the following command to check if the `define_user_attribute` command exists:

```
prompt> define_user_attribute -help
```

Derived User Attributes

Sometimes, existing application attributes might not provide exactly the information that you want. In this case, you can use *derived user attributes* to implement your own attributes in Tcl.

The `define_derived_user_attribute` command allows you to define an attribute whose value is computed by evaluating a Tcl script fragment containing one or more commands:

```
prompt> define_derived_user_attribute \  
        -name frequency \  
        -classes {clock} \  
        -type double \  
        -get_command {expr {1000 / [get_attribute %object period]}}
```

In the Tcl script fragment, the special `%object` key value allows you to access the collection object being queried.

Once defined, the derived user attribute can be queried and filtered with, just like regular application attributes:

```
prompt> get_attribute [all_clocks] frequency
62.5 100 125 125
prompt> get_clocks * -filter {frequency > 100}
rx_clk tx_clk
```

The Tcl script fragment can contain multiple lines. If execution reaches the end of the fragment, the return value of the last command becomes the attribute value. If execution reaches a `return` command, that explicit return value becomes the attribute value:

```
prompt> define_derived_user_attribute \
    -name frequency \
    -classes {clock} \
    -type double \
    -get_command {
        set clock_name [get_attribute %object full_name]
        if {[string match {debug*} $clock_name]} {
            return inf
        }
        expr {1e3 / [get_attribute %object period]}
    }
```

The tool attempts to convert the return value to the data type specified by the `-type` option. An error occurs if the return value is incompatible with the type. Empty-string return values are treated as undefined attribute values, which can be tested for in filter expressions using the `defined()` and `undefined()` attribute functions.

The script fragment is executed in an anonymous namespace. To access variables defined in the calling scope, define them with `variable`. To access global variables (such as application variables), define them with `global` or reference them using the `$::varName` syntax.

The `define_derived_user_attribute` command also provides the following features:

- The `-set_command` and `-remove_command` options allow you to define your own `set_attribute` and `remove_attribute` behaviors, respectively. This allows you to implement custom constructor and destructor behaviors for the attribute, such as storing and removing values in a database.
- The `-subscript_command` option allows you to define your own subscripted attribute behaviors. This allows you to define attributes that are queried in `attribute(subscript)` form.

For more information on these features, see the `define_derived_user_attribute` man page.

To undefine a derived user attribute, use the `undefine_derived_user_attribute` command. This can be useful for temporarily defining a derived user attribute within a Tcl script or procedure.

Using Name-Based Object Specifications

You can reference a design object simply by using the object name. When you invoke a command that operates on objects of different types, the tool searches the design database for an object that matches the specified name.

To explicitly control the types of objects searched, you must use an object search command (`get_*`).

The first command uses implicit object specification; the next command uses explicit object specification.

```
prompt> set_drive 1.0 clk
prompt> set_drive 1.0 [get_ports clk]
```

To avoid object type ambiguity, collection references are recommended over name-based references for commands that accept multiple object types. To determine if this is true for a command, see its man page.

Matching Names of Design Objects During Queries

In synthesis tools, when you query an object, either implicitly or explicitly, you can specify that the tool use a rule-based name matching algorithm to match object names in the netlist with those in memory.

For example, automatic ungrouping by the `compile_ultra` command followed by `change_names` might result in the forward slash (/) separator being replaced with an underscore (_) character. If you enable the rule-based name matching capability, the tool resolves these differences: it can match a cell named `a_b_c/d_e` with the string `a/b_c/d/e`.

To enable the rule-based name matching capability, set the `enable_rule_based_query` variable to `true`. (The default is `false`.) When you enable the variable, the Design Compiler tool uses the rule-based name matching capability with a set of default query rules when it does not find an exact match. Use the `set_query_rules -show` command to display the default query rules.

To customize the query rules, use the `set_query_rules` command. You can do the following:

- Define a list of equivalent hierarchical separators in object names
- Define a list of equivalent bus notation characters

- Specify the object class to which the rule-based name matching rules are applied
- Enable wildcard support

This might match more objects than intended after ungrouping and after the `change_names` command is run.

- Enable the suffix name rule
- Enable case-insensitive rule-based matching
- Report the current rule-based name matching rules
- Display the default query rules
- Report messages indicating that the object is matched using query rules
- Reset the query rules to the program default

Runtime might be much slower when you use rule-based name matching. You should disable it when it is no longer needed.

In the following example, case-insensitive rule-based name matching rules are applied to the cell, port, and pin object classes for the b.in_a/c.in_b cells:

```
prompt> set_app_var enable_rule_based_query true
Information: Rule-based matching enabled. Please turn it off once
rule-based matching is no longer needed. (UID-1056)
true
prompt> set_query_rules -class {cell pin port} -nocase -show
*****
Query Rules (Enabled)
*****
          Rule Type      Rule Value
-----
Hierarchy-Separator    {/_ _ .}
      Bus-Notation      {[] _ ()}
      Object-Class      {cell port pin}
          Nocase        true
          Wildcard       false
          Verbose        false
1
prompt> get_cells  b_In/a_C/iN.b
{b.in_a/c.in_b}
prompt> set_app_var enable_rule_based_query false
false
```

For details about the available options, see the `set_query_rules` man page.

Reporting Collections

The `report_collection` command allows you to create detailed, customized reports for the contents of a collection.

By default, the command reports the name and object type for each item in the collection. For example,

```
prompt> set my_fanin [all_fanin -flat -startpoints_only -to ...]
...
prompt> report_collection $my_fanin
...
name                object_class
-----
CLK                  port
Uclkdiv1_reg/Q       pin
Uclkdiv2_reg/Q       pin
Uclkdiv3_reg/Q       pin
sel[0]               port
sel[1]               port
```

To customize the report, you can specify the list of attributes to display in the report by using the `-columns` option. Nested attributes are permitted. For example,

```
prompt> set my_points [get_attribute $my_path points]
...
prompt> report_collection $my_points \
    -columns {object object.cell.ref_name arrival transition}
...
object                object.cell.ref_name  arrival  transition
-----
my_reg1/CLK           fd1a1                0.000000  0.060002
my_reg1/Q              fd1a1                0.773803  0.043376
U1/A                   bufla1               0.792937  0.058122
U1/Y                   bufla1               1.039227  0.035672
Q                      bufla1               1.039227  0.035672
```

Undefined attribute values result in blank entries with no warnings.

You can include attributes that return collection values. Single-item values return that object's name, while multiple-item values return a summary of the objects. For example,

```
prompt> report_collection [all_registers -clock_pins] \
    -columns {full_name clocks}
...
full_name              clocks
-----
Uclkdiv1_reg/CLK       CLK
Uclkdiv2_reg/CLK       CLK_div2
Uclkdiv3_reg/CLK       CLK_div4
```

```
my_reg1/CLK          clock:4
my_reg2/CLK          clock:4
```

The `-type` option of the `report_collection` command allows you to specify what type of information to report within each attribute column. You can specify a list of one or more of the following types:

- `values`: Report the set of attribute values by collection item (default)
- `statistics`: Report statistical information across values for numerical attributes
- `distribution`: Report information about the distribution of attribute values

For example, to show statistical information about the clock pins in a design,

```
prompt> report_collection [all_registers -clock_pins] \
        -columns {actual_transition_max net.number_of_leaf_loads} \
        -type {statistics}
...

```

	actual_transition_max	net.number_of_leaf_loads
Minimum :	0.091904	11.000000
Lower Quartile (25%) :	0.121821	19.000000
Median (50%) :	0.125916	20.000000
Upper Quartile (75%) :	0.129933	21.000000
Maximum :	0.176954	30.000000
Mean :	0.127152	19.952200
Standard Deviation :	0.010409	2.121478
Null Value Count :	0	0
Valid Value Count :	10502	10502
Unique Value Count :	211	18

To show distribution information for the types of register cells in a design,

```
prompt> report_collection [all_registers] \
        -columns {ref_name} \
        -type {distribution}
...

```

	ref_name
Null Value Count :	0
Valid Value Count :	10502
Unique Value Count :	21
Value Distribution :	EDFFX1 (8820)
	DFFRX1 (918)
	EDFFX4 (329)
	DFFRHQX1 (137)
	EDFFXL (79)
	DFFSHQX1 (61)
	SDDFFRHQX1 (39)

```
DFFSX1      ( 38)
DFFRX2      ( 36)
DFFHQX1     ( 9)
DFFRHQX4    ( 9)
DFFRX4      ( 9)
DFFRXL      ( 6)
DFFSHQXL    ( 3)
EDFFX2      ( 2)
JKFFRXL     ( 2)
DFFRHQX2    ( 1)
DFFTRX1     ( 1)
DFFX1       ( 1)
SDFFRHQXL   ( 1)
SDFFSHQX1   ( 1)
```

You can use the Tcl `alias` command to create aliases for your favorite reports. Place them in your tool's setup file to make them available in every tool session.

The `report_collection` command provides many more features to control the processing and formatting used for the report. For more information, see the man page.

Writing Collection Information to a File

The `write_collection` command allows you to write comma-separated or tab-separated attribute value information about a collection to a file.

By default, the command writes the name and object type of each collection item, in the order of the input collection, to a comma-separated value (CSV) file. For example,

```
prompt> set my_fanin [all_fanin -flat -startpoints_only -to ...]
...
prompt> write_collection $my_fanin -file out.txt
prompt> sh cat out.txt
name,object_class
CLK,port
Uff1/Q,pin
Uff2/Q,pin
Uff3/Q,pin
sel[0],port
sel[1],port
```

To write the information in tab-separated value (TSV) format instead, use the `-format tsv` option:

```
prompt> write_collection $my_fanin -file out.txt -format tsv
prompt> sh cat out.txt
name      object_class
CLK       port
Uff1/Q    pin
Uff2/Q    pin
```

```
Uff3/Q  pin
sel[0]  port
sel[1]  port
```

To customize the output, you can specify the list of attributes to display in the report by using the `-columns` option. Nested attributes are permitted. For example,

```
prompt> set my_points [get_attribute $my_path points]
...
prompt> write $my_points -file out.txt \
    -columns {object object.cell.ref_name arrival transition}
prompt> sh cat out.txt
object,object.cell.ref_name,arrival,transition
D,,0.000000,0.000000
U1/A,bufla1,0.000000,0.000000
U1/Y,bufla1,0.238213,0.028706
Ufoo_reg/D,fdla1,0.238213,0.028706
```

Undefined attribute values result in blank entries with no warnings.

You can include attributes that return collection values. Single-item values return that object's name, while multiple-item values return a summary of the objects. For example,

```
prompt> write_collection [all_registers -clock_pins] -file out.txt \
    -columns {full_name clocks}
prompt> sh cat out.txt
full_name,clocks
Uff1/CLK,CLK
Uff2/CLK,CLK_div2
Uff3/CLK,CLK_div4
Ufoo_reg/CLK,clock:4
```

The `write_collection` command provides additional options to control the output file format. For more information, see the man page.

7

Using Scripts

A command script (script file) is a sequence of commands in a text file. Command scripts enable you to execute commands automatically. A command script can start the command-line interface, perform various processes on your design, save the changes by writing them to a file, and exit the session. You can use scripts interactively from the command line or call scripts from within other scripts.

You can create and modify scripts by using a text editor. You can also use the `write_script` command to create new scripts from existing scripts.

This chapter includes the following sections:

- [Using Command Scripts](#)
- [Creating Scripts](#)
- [Using the Output of the `write\_script` Command](#)
- [Running Command Scripts](#)

Using Command Scripts

Command scripts help you in several ways. Using command scripts, you can

- Manage your design database more easily
- Save the attributes and constraints used during the design process in a script file and use the script file to restart the design flow
- Create script files with the defaults you want to use and use these within other scripts
- Include constraint files in scripts (constraint files contain the commands used to constrain the modules to meet your design goals)

Additionally, Synopsys provides checkers for checking scripts for syntax and context errors.

You can keep a frequently used set of commands in a script file and reuse them in a later session.

You can write comments in your script file. Start the comment line with the pound sign (#). You can create inline comments by placing a semicolon between a command and the pound sign. For example,

```
echo abc; # this is an inline comment
```

When the command continuation character (\) is placed at the end of a commented command line, the subsequent line is also treated as a comment.

For an example script, see [A Tcl Script Example](#).

Creating Scripts

You can create a script in several ways:

- Write a command history

For more information, see [Running Linux Commands Within the Tool](#).

- Write scripts manually
- Edit the command log file

You can customize the name of this log file by setting the `command_log_file` variable.

Using the Output of the `write_script` Command

Use the `write_script` command to build new scripts from existing scripts.

To build new scripts from existing scripts,

1. Use the redirection operator (>) to redirect the output of `write_script` to a file.
2. Edit the file as needed.
3. Rerun the script to see new results.

For example, to save constraints on the design to the `test.tcl` file, use the following command:

```
prompt> write_script > test.tcl
```

Running Command Scripts

Run a command script in one of two ways:

- From within the tool, use the `source` command to execute the script file
- When you invoke the tool, use the `-f` option to execute the script file

Running Scripts From Within the Tool

The `source` command executes a command script from within the tool. Use the `source` command on the command line or within a script file. Use the *file* argument to specify the name of the script file. If *file* is a relative path name, the tool scans for the file in the directories listed in the `search_path` variable and reads the file from the first directory in which it exists.

By default, the tool does not display commands in the script file as they execute. To display the commands as they are processed, use the `-echo` and `-verbose` options to the `source` command.

For example, to execute the commands contained in the `my_script` file in your home directory, use the following command:

```
prompt> source -echo -verbose ~/my_script
command
# comment
command
...
```

You can also save the execution results to an output file by using the redirection operator (`>`). For example,

```
prompt> source -echo -verbose myrun.tcl > myrun.out
```

The execution output of a script file can be changed in various ways. For example, you can change how variable initializations and error and warning messages are displayed. For more information about controlling execution output, see the man pages for the `sh_new_variable_message` and `suppress_message` commands.

Note:

The Synopsys implementation of the `source` command varies from the Tcl implementation. For `source` usage information, see the Synopsys man pages.

Running Scripts During Tool Invocation

The tool invocation command with the `-f` option executes a script file before displaying the initial prompt.

The syntax is

```
tool_invocation_cmd -f script_file
```

If the last statement in the script file is `quit`, no prompt appears and the command shell exits.

For example, to run the `common.tcl` script file when you start `dc_shell` and redirect the commands and error messages to a file named `output_file`, use the following command:

```
% dc_shell -f common.tcl >& output_file
```


8

A Tcl Script Example

This chapter contains an example script that demonstrates how to use many of the commands and topics covered in previous chapters. The various aspects of the example script are described in detail.

The example script contains the `rpt_cell` procedure and the `define_proc_attributes` command, which is used to extend the attributes of the `rpt_cell` procedure.

This chapter contains the following sections:

- [rpt_cell Overview](#)
- [rpt_cell Listing and Output Example](#)
- [rpt_cell Details](#)

rpt_cell Overview

The `rpt_cell` script has two components. The first is the `rpt_cell` procedure; the second is the `define_proc_attributes` command. The `define_proc_attributes` command extends the attributes of the `rpt_cell` procedure.

The `rpt_cell` procedure lists all cells in a design and reports if a cell has the following properties:

- Is a black box (unknown)
- Has a don't touch attribute
- Is hierarchical
- Is combinational
- Is a test cell

The `rpt_cell` procedure takes one argument. The argument is treated as an option that specifies a desired report type. The options are,

- `-all_cells` – Reports one line per cell, and it generates a summary of the cell count.
- `-hier_only` – Reports only the hierarchical blocks, and it generates a summary of the cell count.
- `-total_only` – Displays only a summary of the cell count.

The `define_proc_attributes` command is placed after the `rpt_cell` procedure in the `rpt_cell` script file. This command is used to provide help information about the `rpt_cell` procedure. The help information is used in conjunction with the `help` command and includes a short description of the `rpt_cell` procedure and its options.

A full listing of the `rpt_cell` script and an example of the output from the `rpt_cell` script are shown in [rpt_cell Listing and Output Example](#).

To use the `rpt_cell` script, enter or copy it into a text file named `rpt_cell.tcl`, load it into the Synopsys shell by using the `source` command, and then load a design database. The syntax for the `rpt_cell` procedure is

```
rpt_cell arg
```

For example,

```
prompt> source rpt_cell.tcl
prompt> read_file -format ddc TLE_mapped.ddc
prompt> rpt_cell -total_only
```

rpt_cell Listing and Output Example

[Example 4](#) shows the full listing of the `rpt_cell` example script file.

Example 4 *rpt_cell.tcl Listing*

```
#Title:          rpt_cell.tcl
#
#Description: This Tcl procedure generates a cell
#             report of a design.
#             It reports all cells and the following attributes:
#             b - black box (unknown)
#             d - has dont_touch attribute
#             h - hierarchy
#             n - noncombinational
#             t - test cell
#
#Options:        -all_cells    one line per cell plus summary
#               -hier_only    every hierarchy cell and summary
#               -total_only   generate summary only
#
#Usage:         prompt> source rpt_cell.tcl
```

Chapter 8: A Tcl Script Example

rpt_cell Listing and Output Example

```

#           prompt> rpt_cell -t
#
proc rpt_cell args {
    suppress_message UID-101

    set option [lindex $args 0]
    if {[string match -a* $option]} {
        echo ""
        echo "Attributes:"
        echo " b - black-box (unknown)"
        echo " d - dont touch"
        echo " h - hier"
        echo " n - noncombo"
        echo " t - test cell"
        echo ""
        echo [format "%-32s %-14s %5s %11s" "Cell" "Reference" "Area" "Attribut
es"]
        echo "-----"
    } elseif {[string match -t* $option]} {
        set option "-total_only"
        echo ""
        set cd [current_design]
        echo "Performing cell count on [get_object_name $cd] ..."
        echo ""
    } elseif {[string match -h* $option]} {
        set option "h"; # hierarchical only
        echo ""
        set cd [current_design]
        echo "Performing hierarchical cell report on [get_object_name $cd] .
.."
        echo ""
        echo [format "%-36s %-14s %11s" "Cell" "Reference" "Attributes"]
        echo "-----"
    } else {
        echo ""
        echo " Message: Option Required"
        echo " Usage: rpt_cell \[-all_cells\] \[-hier_only\]
\[-total_only\]"
        echo ""
        return
    }

    # initialize summary vars
    set total_cells 0
    set dt_cells 0
    set hier_cells 0
    set hier_dt_cells 0
    set seq_cells 0
    set seq_dt_cells 0
    set test_cells 0
    set total_area 0

    # initialize other vars
    set hdt ""
    set tc_atr ""
    set xc_cell_area 0

    # create a collection of all cell objects
    set all_cells [get_cells -hierarchical *]

```

Chapter 8: A Tcl Script Example

rpt_cell Listing and Output Example

```

foreach_in_collection cell $all_cells {
    incr total_cells

    set cell_name [get_attribute $cell full_name]
    set dt [get_attribute $cell dont_touch]

    if {$dt=="true"} {
        set dt_atr "d"
        incr dt_cells
    } else {
        set dt_atr ""
    }

    set ref_name [get_attribute $cell ref_name]
    set cell_area [get_attribute $cell area]

    if {$cell_area > 0} {
        set xcell_area $cell_area
    } else {
        set cell_area 0
    }

    set t_cell [get_attribute $cell is_a_test_cell]
    if {$t_cell=="true"} {
        set tc_atr "t"
        incr test_cells
    } else {
        set tc_atr ""
    }

    set hier [get_attribute $cell is_hierarchical]
    set combo [get_attribute $cell is_combinational]
    set seq [get_attribute $cell is_sequential]

    if {$hier} {
        set attribute "h"
        incr hier_cells
        set hdt [concat $option $hier]
        if {$dt_atr=="d"} {
            incr hier_dt_cells
        }
    } elseif {$seq} {
        set attribute "n"
        incr seq_cells
        if {$dt_atr=="d"} {
            incr seq_dt_cells
        }
    }
    set total_area [expr $total_area + $xcell_area]
    } elseif {$combo} {
        set attribute ""
        set total_area [expr $total_area + $xcell_area]
    } else {
        set attribute "b"
    }
}

if {[string match -a* $option]} {
    echo [format "%-32s %-14s %5.2f %2s %1s %1s" $cell_name $ref_name \
        $cell_area $attribute $dt_atr $tc_atr]
} elseif {$hdt=="h true"} {
    echo [format "%-36s %-14s %2s" $cell_name $ref_name $attribute \
        $dt_atr]
    set hdt ""
}

```

Chapter 8: A Tcl Script Example

rpt_cell Listing and Output Example

```

    } ; # close foreach_in_collection

    echo "-----"
    echo [format "%10s Total Cells" $total_cells]
    echo [format "%10s Cells with dont_touch" $dt_cells]
    echo ""
    echo [format "%10s Hierarchical Cells" $hier_cells]
    echo [format "%10s Hierarchical Cells with dont_touch" $hier_dt_cells]
    echo ""
    echo [format "%10s Sequential Cells (incl Test Cells)" $seq_cells]
    echo [format "%10s Sequential Cells with dont_touch" $seq_dt_cells]
    echo ""
    echo [format "%10s Test Cells" $test_cells]
    echo ""
    echo [format "%10.2f Total Cell Area" $total_area]
    echo "-----"
    echo ""
}

define_proc_attributes rpt_cell \
    -info "Procedure to report all cells in the design" \
    -define_args {
        {-a "report every cell and the summary"}
        {-h "report only hierarchical cells and the summary"}
        {-t "report the summary only"} }

```

Example 5 shows an output example from the `rpt_cell` procedure, using the `-h` (`-hier_only`) option.

Example 5 `rpt_cell` Output Example

Current design is 'TLE'.
Performing hierarchical cell report on TLE ...

Cell	Reference	Attributes
datapath	fast_add8	h
Multiplicand_reg	reg8	h
control_unit	control	h
Op_register	super_reg17	h
datapath/CLA_0	CLA_4bit_1	h
datapath/CLA_1	CLA_4bit_0	h
datapath/CLA_0/FA_0	full_adder_7	h
datapath/CLA_0/FA_1	full_adder_6	h
datapath/CLA_0/FA_2	full_adder_5	h
datapath/CLA_0/FA_3	full_adder_4	h
datapath/CLA_1/FA_0	full_adder_3	h
datapath/CLA_1/FA_1	full_adder_2	h
datapath/CLA_1/FA_2	full_adder_1	h
datapath/CLA_1/FA_3	full_adder_0	h

```

    247 Total Cells
      0 Cells with dont_touch
    14 Hierarchical Cells

```

```
0 Hierarchical Cells with dont_touch
32 Sequential Cells (incl Test Cells)
0 Sequential Cells with dont_touch
0 Test Cells
663.00 Total Cell Area
-----
```

rpt_cell Details

The `rpt_cell` script is described sequentially in the following sections:

- [Defining the Procedure](#)
- [Suppressing Warning Messages](#)
- [Examining the Procedure Argument](#)
- [Initializing Variables](#)
- [Creating and Iterating Over a Collection](#)
- [Collecting the Report Data](#)
- [Formatting the Output](#)

Defining the Procedure

The `rpt_cell` procedure requires only one argument, so its definition is simple.

[Example 6](#) shows how `rpt_cell` is defined.

Example 6 rpt_cell proc Definition

```
proc rpt_cell args {
    procedure body ...
}
```

You use the `proc` command to define the procedure; `rpt_cell` is the name of the procedure, and `args` is the variable that receives all command line arguments when the procedure is invoked. The value of `args` is used later within the body of the procedure, as described in [Examining the Procedure Argument](#).

The `define_proc_attributes` command provides additional (extended) information about a procedure, which is used in conjunction with the `help` command. See [Using the define_proc_attributes Command](#). [Example 7](#) shows how the `define_proc_attributes` command is used with the `rpt_cell` procedure.

Example 7 `define_proc_attributes` Command

```
define_proc_attributes rpt_cell \  
-info "Procedure to report all cells in the design" \  
-define_args {  
    {-a "report every cell and the summary"}  
    {-h "report only hierarchical cells and the summary"}  
    {-t "report the summary only"} }  
}
```

The additional information consists of a one-line description of the `rpt_cell` procedure and descriptions of the options it expects. [Example 8](#) shows an example of the `help` command displayed for the `rpt_cell` procedure. To see argument information with the `help` command, use the `-verbose` option.

Example 8 `rpt_cell` Help Usage

```
prompt> help -verbose rpt_cell  
rpt_cell      # Procedure to report all cells in the design)  
-a            (report every cell and the summary)  
-h            (report only hierarchical cells and the summary)  
-t            (report the summary only)
```

Suppressing Warning Messages

The first line within the body of the `rpt_cell` procedure, shown in [Example 9](#), is used to suppress UID-101 warning messages that occur when an attribute-related command does not find a given attribute.

Example 9 `suppress_message` Command

```
proc rpt_cell args {  
    suppress_message UID-101  
    ...  
}
```

The `rpt_cell` procedure reports information about specific cell attributes; however, some of the cells within the design might not have one of these specific attributes. If this situation occurs repeatedly, a large number of warning messages is generated and output to the screen, or if you redirect the output to a log file, the log file might become undesirably large. Because a UID-101 warning message does not affect the meaning of the report and is likely to occur frequently within the `rpt_cell` procedure, it is suppressed.

You use the `suppress_message` command to disable the printing of a specific warning or informational message. For more information, see the `suppress_message` man page.

Examining the Procedure Argument

The section of script shown in [Example 10](#) extracts the report type option from the procedure argument and uses this value to determine what the report header looks like; furthermore, this section is used to handle the entry of invalid options.

Example 10 Examining the Procedure Argument

```
...
set option [lindex $args 0]
if {[string match -a* $option]} {
    ...
} elseif {[string match -t* $option]} {
    ...
} elseif {[string match -h* $option]} {
    ...
} else {
    ...
}
...
```

The argument to the `rpt_cell` procedure is used to specify what type of report to generate. The `lindex` command is used to extract the option from the `args` variable, and the result is placed into the `option` variable. The `string` command with its `match` option is then used to conditionally determine what the report header looks like.

The report options are `-all_cells`, `-hier_only`, or `-total_only`; however, the values `-a`, `-h`, and `-t` are all that are required because the wildcard character (*) is used in the `string match` command. (For more information, see the `string` man page.)

The `echo` command is used to output information, and the `format` command is used in conjunction with the `echo` command to generate formatted output. See [Example 11](#).

Example 11 echo and format Commands

```
...
echo "Performing hierarchical cell report on [get_object_name
$cd] ..."
echo " "
echo [format "%-36s %-14s %11s" "Cell" "Reference" "Attributes"]
...
```

You use the `format` command to format lines of output in the same manner as the C `sprintf` procedure. The use of the `format` command within the `rpt_cell` procedure is described in more detail in [Formatting the Output](#).

The `current_design` and `get_object_name` commands are used to display the name of the current design. See [Example 12](#).

Example 12 current_design and get_object_name Commands

```
...
} elseif {[string match -t* $option]} {
    set option "-total_only"
    ...
    set cd [current_design]
    echo "Performing cell count on [get_object_name $cd] ..."
    ...
} elseif {[string match -h* $option]} {
    set option "h"; # hierarchical only
    echo ""
}
```



```

        set cd [current_design]
        echo "Performing hierarchical cell report on [get_object_name $cd] .
    .."
    ...

```

You use the `current_design` command to set the working design; however, if used without arguments, it returns a collection containing the current working design. This collection is then passed to the `get_object_name` command to obtain the name of the current design.

Note how the following line of the script (from [Example 12](#)) is constructed:

```
set option "h";    # hierarchical only
```

In Tcl, you can place multiple commands on one line by using a semicolon to separate the commands. You can use this feature as a way to form inline comments.

The `else` block (see [Example 13](#)) handles an invalid option condition. If no option or an invalid option is specified, the procedure prints out a message that shows proper argument usage and then exits.

Example 13 Invalid Option Message

```

    ...
} else {
    echo " "
    echo "    Message: Option Required"
    echo "    Usage: rpt_cell \[-all_cells\] \[-hier_only\] \[-total_only\]"
    echo " "
    return
}
    ...

```

Initializing Variables

The section of the script shown in [Example 14](#) uses the `set` command to initialize some of the variables used by the `rpt_cell` procedure.

Example 14 Variable Initialization

```

    ...
    # initialize summary vars
    set total_cells 0
    set dt_cells 0
    set hier_cells 0
    set hier_dt_cells 0
    set seq_cells 0
    set seq_dt_cells 0
    set test_cells 0
    set total_area 0

    # initialize other vars
    set hdt ""

```

```
set tc_atr ""
set xcell_area 0
...
```

The values for these particular variables are expected to change within the `foreach_in_collection` loop and within `if` blocks that might not be executed, so these variables are set to 0 here to prevent a “no such variable error” should the loop or `if` blocks not be executed.

Creating and Iterating Over a Collection

A collection is used to hold the list of all cells in the design. Then, a `foreach_in_collection` loop is used to obtain the attribute information about each cell and to cumulate results for the summary section of the report. [Example 15](#) shows the command used to create the collection and the `foreach_in_collection` loop.

Example 15 Collection Iteration

```
...
set all_cells [get_cells -hierarchical *]
foreach_in_collection cell $all_cells {
    ...
}; # close foreach_in_collection
...
```

The `get_cells` command creates a collection of cells from the current design. The `-hierarchical` option tells `get_cells` to search for cells level by level. The wildcard character (*) is used as the pattern name to match—in this case, all cell names.

The result of the `get_cells` command is saved in the `all_cells` variable. This variable is then used by the `foreach_in_collection` command to iterate over all the objects in the collection. For each iteration, an object is placed in the `cell`, which is used in the body of the `foreach_in_collection` block to derive information about that object (cell name, reference name, cell area, and cell attributes).

Collecting the Report Data

The report data is collected into a set of variables by the `foreach_in_collection` loop shown in [Example 15](#). Cell information is obtained from the design database by the `get_attribute` command, and the summary data is cumulated inside of `if` blocks at various locations within the `foreach_in_collection` loop. [Table 18](#) lists the variables used for the report.

Table 18 *rpt_cell Report Variables*

Variable	Description
Variables used in the main body of the report	
cell_name	Cell name
ref_name	Reference name
cell_area	Cell area
attribute	Cell's attribute
dt_atr	Don't touch attribute
tc_atr	Test cell attribute
Variables used in the summary section of the report	
total_cell	Total number of cells
dont_touch	Number of cells with don't touch attribute
hier_cells	Number of hierarchical cells
hier_dt_cells	Number of hierarchical cells with don't touch attribute
seq_cells	Number of sequential cells (includes test cells)
seq_dt_cells	Number of sequential cells with don't touch attribute
test_cells	Number of test cells
total_area	Total cell area

The body of the `foreach_in_collection` loop looks complex, but the pseudocode shown in [Example 16](#) shows how straightforward it really is.

Example 16 *Body of foreach_in_collection Loop*

```
...
foreach_in_collection cell $all_cells {
  - Cumulate total cell count
  - Get cell name
  - Collect don't touch attribute information
  - Get reference name of cell
  - Get cell area
  - Collect test cell attribute information
  - Collect hierarchical attribute information
  - Collect combinational attribute information
  - Collect sequential attribute information
}
```

```

- Cumulate total area
- Output one line of cell information
- Return to top of loop and process next cell object
}; # close foreach_in_collection
...

```

You obtain cell attributes from the design database by using the `get_attribute` command, as shown in [Example 17](#).

Example 17 Obtaining Cell Attributes

```

...
set dt [get_attribute $cell dont_touch]
...
set ref_name [get_attribute $cell ref_name]
set cell_area [get_attribute $cell area]
...
set t_cell [get_attribute $cell is_a_test_cell]
...
set hier [get_attribute $cell is_hierarchical]
set combo [get_attribute $cell is_combinational]
set seq [get_attribute $cell is_sequential]
...

```

Attributes are properties assigned to design objects, and they range in values. Some are predefined values, like `dont_touch`; others are user-defined, while still others can be logical in nature and have values such as `true` or `false`. You can find detailed information about object properties in the attributes man pages.

The `if` blocks are used to determine whether the cell has one or more of the properties: don't touch, test cell, hierarchical, sequential, or combinational. Along the way, the totals for the summary section of the report are cumulated. [Example 18](#) shows an example `if` block.

Example 18 if Block Example

```

...
set dt [get_attribute $cell dont_touch]

if {$dt=="true"} {
    set dt_atr "d"
    incr dt_cells
} else {
    set dt_atr ""
}
...

```

This `if` block determines whether the cell has the `dont_touch` attribute, and if so, it sets the don't touch attribute variable `dt_atr` to `d` and increments the count of don't touch cells (`dt_cells`). If the cell does not have the `dont_touch` attribute, the `dt_atr` variable is set

to null. The other `if` blocks in the body of the `foreach_in_collection` loop work in a similar way.

One line of cell information is printed at the end of the `foreach_in_collection` loop. The script that handles this step is shown in [Example 19](#).

Example 19 Cell Information Output

```
if {[string match -a* $option]} {  
    echo [format "%-32s %-14s %5.2f %2s %1s %1s" \  
        $cell_name $ref_name $cell_area $attribute $dt_atr $tc_atr]  
} elseif {$hdt=="h true"} {  
    echo [format "%-36s %-14s %2s" \  
        $cell_name $ref_name $attribute $dt_atr]  
    ...  
}
```

There are two possible formats for the line of output; an `if` block is used to handle the two possibilities. The line of output is formatted by the `format` command. How the `format` command is used by the `rpt_cell` procedure is explained in the next section.

After a line of cell information is output, the next cell object is processed.

Formatting the Output

This section provides an overview of the `format` command as it is used by the `rpt_cell` procedure. The options of the `format` command are extensive; see the `format` man page for a complete description.

[Example 20](#) shows an output example from the `rpt_cell` procedure.

Example 20 rpt_cell Output Example

```
Current design is 'TLE'.  
Performing hierarchical cell report on TLE ...  
  
Cell                                Reference  Attributes  
-----  
datapath                            fast_add8      h  
Multiplicand_reg                    reg8          h  
  
...  
  
datapath/CLA_1/FA_2                 full_adder_1   h  
-----  
247 Total Cells  
0 Cells with dont_touch  
  
...
```

```
663.00 Total Cell Area
```

Each line of output is generated by the `echo` command. Formatted output is handled by the `format` command in conjunction with the `echo` command.

The basic form of the `format` command is

```
format format_string arg_list
```

The *format_string* argument contains text and conversion specifiers. The *arg_list* argument contains one or more variables that are to be substituted into the conversion specifiers. For example, the following command is used in the summary section of the `rpt_cell` report:

```
echo [format "%10s Total Cells" $total_cells]
```

In this example, the value of the `total_cells` variable is substituted into the conversion specifier, `%10s`, and is formatted according to the conversion specifier. In this case, the `total_cells` value is converted into a text string that is 10 characters wide.

There is a one-to-one correspondence between conversion specifiers and the variables placed in the argument list. For example,

```
echo [format "%-32s %-14s %5.2f %2s %1s %1s" \  
    $cell_name $ref_name $cell_area $attribute $dt_atr $tc_atr]
```

In this example, the list of variables is paired with each of the `format` specifiers.

The components of the conversion specifier can be used to specify conversion properties such as data type, minimum field width, precision, and field justification. For example, `%5.2f` specifies conversion of a floating point number to a text string that has five characters to left of the decimal point and two characters to the right.

9

Command-Line Editor

The command-line editor allows you to use common keyboard shortcuts found in Linux terminals and the VI or Emacs editor to simplify the editing of command-line text and quickly recall previous commands.

Note:

This product includes software developed by the University of California, Berkeley and its contributors.

To learn how to use the command-line editor, see

- [Changing the Settings of the Command-Line Editor](#)
- [Listing Key Mappings](#)
- [Setting the Key Bindings](#)
- [Navigating the Command Line](#)
- [Completing Commands, Variables, and File Names](#)
- [Searching the Command History](#)

Changing the Settings of the Command-Line Editor

The command-line editor is enabled by default. You can disable the command-line feature by setting the `shell.common.enable_line_editing` application option to `false`. You can use the `set_cle_options` command to change the default settings, as described in the following table:

To do this	Use this
Set the key bindings (default is <code>emacs</code> editing mode)	<code>-mode vi emacs</code>
Set the terminal beep (default is off)	<code>-beep on off</code>
Specify default settings	<code>-default</code>

If you enter `set_cle_options` without any options, the current settings are displayed.

Listing Key Mappings

The command-line editor allows you to access any of the last 1000 commands by using a combination of keys. In addition, you can manipulate text on the command line and kill and yank text. Killing (or cutting) text is the process by which text is deleted from the current line but saved for later use. Yanking (or pasting) text is the process by which the deleted text is reinserted into the line.

These features are available in both vi and emacs mode. For a complete list of key mappings, use the `sh_list_key_bindings` command.

Note:

In the key mappings displayed when you use the `sh_list_key_bindings` command, the text Ctrl+K is the character that results when you press the Ctrl key together with the K key.

Meta+K is the character that results when you press the Meta key together with the K key. On many keyboards, the Meta key is labeled Alt. On keyboards with two Alt keys, the one on the left of the Space bar is generally set as the Meta key. The Alt key on the right of the Space bar might also be configured as the Meta key or some other modifier, such as Compose, which is used to enter accented characters.

If your keyboard does not have a Meta or Alt key or any other key configured as a Meta key, press Esc followed by the K key (for Meta+K). This is known as “metafying” the K key.

Setting the Key Bindings

By default, the key bindings are set to emacs editing mode. To change the key bindings to vi mode, use the `-mode` option of the `set_cle_options` command. You can also use the `sh_line_editing_mode` variable to change the key bindings to vi mode. You can set this variable in either the setup file or at the shell prompt.

The following commands show various methods of setting the editing mode.

```
prompt> set_cle_options -mode emacs
Information: Command line editor mode is set to emacs
successfully. (CLE-01)

prompt> set sh_line_editing_mode vi
Information: Command line editor mode is set to vi
successfully. (CLE-01)
vi

prompt> set sh_line_editing_mode abc
Error: Command line editor mode cannot be set to 'abc'.
```



```
Proceeding with vi mode. (CLE-02)
vi
```

Navigating the Command Line

Use the keys listed in [Table 19](#) to navigate the command line in both vi and emacs mode.

Table 19 *Command-Line Navigation Keys*

Key	Action
Down	Moves the cursor down to the next command.
Up	Moves the cursor up to the previous command.
Left	Moves the cursor to the previous character.
Right	Moves the cursor to the next character.
Home	Moves the cursor to the start of the current line.
End	Moves the cursor to the end of the line.

Completing Commands, Variables, and File Names

You can press the Tab key to complete commands automatically (including nested commands) and their options, variables, and file names. Additionally, you can use the Tab key to automatically complete aliases (short forms for the commands you commonly use, defined with the `alias` command). When removing alias definitions by using the `unalias` command, you can use the command completion feature to list alias definitions.

In all these cases, the results are sorted alphabetically; if the command-line editor cannot find a matching string, it lists all closely matching strings.

[Table 20](#) lists the results of pressing the Tab key within different contexts.

Table 20 *Result of Pressing the Tab Key Within Different Contexts*

Context	Action taken by the command-line editor
Command is not entered fully	Completes the command.
Command is followed by a hyphen (-)	Completes the command argument.
After a > or command	Completes the file name.
After a set, unset, or printvar command	Completes the variable.

Table 20 Result of Pressing the Tab Key Within Different Contexts (Continued)

Context	Action taken by the command-line editor
After a dollar sign (\$)	Completes the variable.
After the help command	Completes the command.
After the man command	Completes the command or variable.
In all other contexts	Completes file names.

Searching the Command History

The command-line editor provides an incremental search capability. You can search the command history in both vi and emacs mode by pressing Ctrl+R. A secondary prompt appears below the command prompt. The command-line editor searches the history as you enter each character of the search string and displays the first matching command. You can continue to add characters to the search string if the matching command is not the one you are searching for. As long as the search string is valid, a colon (:) appears in the secondary search prompt; otherwise a question mark (?) appears in the secondary search prompt and the command-line editor retains the last successful match on the prompt. After you find the command that you are searching for, you can press the Return key to execute the command.